

الجمهورية العربية السورية

وزارة التعليم العالي

الجامعة السورية الخاصة



الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

نظام إدارة المراكز الصحية

مشروع (تخرج 1) - قدم استكمالاً لمتطلبات الحصول على درجة البكالوريوس في
هندسة المعلوماتية هندسة البرمجيات ونظم المعلومات الذكية

إعداد

عبدالرحمن البردان حسين فخر الدين عبد النبي ايمن صليبي

إشراف

د. شادي بليدي

م. مايا الاحمد

Contents

6	الفصل الأول : المقدمة
7	1.1 المقدمة
7	1.2 المراكز الصحية
7	1.3 تحديد المشكلة
7	1.4 فكرة المشروع
7	1.5 هدف المشروع
7	1.6 الحل المقترح للمشكلة
8	1.7 حدود النظام
8	1.8 الخلاصة
9	الفصل الثاني : المفاهيم الأساسية والدراسة المرجعية
10	2.1 منهجية التطوير المختارة والمبررات
11	2.2 مخطط جانت (Gantt Chart)
11	2.3 إدارة المخاطر
12	2.4 فريق العمل وتوزيع المهام والأدوار
12	2.5 إدارة الجودة في المشروع
12	2.6 أدوات إدارة المشروع المستخدمة
13	2.7 آليات المراقبة والتقييم المستمر
13	2.8 إدارة تشكيلة البرمجيات SCM
14	الفصل الثالث : الدراسة التحليلية للنظام
15	3.1 الدراسات السابقة في مجال الأنظمة الصحية الإلكترونية
15	3.1.1 نظام إدارة المستشفى الإلكتروني - Ridhab (2018)
15	3.1.2 نظام إدارة المستشفى المبني على الويب - Rahman (2022)
15	3.1.3 نظام إدارة المستشفى - Tuhin (2023)
16	3.2 التحليل المقارن للأنظمة الحالية
16	3.3 الفجوة البحثية والابتكار المقترح
17	3.4 التقنيات والأطر الحديثة في المجال
18	3.5 الإطار النظري والدراسات المرجعية
18	3.6 التوجهات الحديثة والمستقبلية في المجال
19	الفصل الرابع: نمذجة وتحليل المتطلبات
20	4.1 دراسة الجدوى
20	4.1.1 الجدوى الفنية

20	4.1.2 الجدوى التشغيلية
20	4.1.3 الجدوى الاقتصادية
20	4.2 تحديد أصحاب المصلحة وتحليل احتياجاتهم
21	4.3 المتطلبات الوظيفية (Functional Requirements)
22	4.4 المتطلبات غير الوظيفية (Non-Functional Requirements)
22	4.5 قائمة أولويات المتطلبات (MoSCoW)
23	4.6 مخططات UML للمتطلبات
23	4.6.1 حالات الاستخدام الرئيسية (Use Cases)
24	4.6.2 توصيف حالات الاستخدام (Use Case Descriptions)
24	أولاً: الصيدلي (Pharmacist)
29	ثانياً: المدير – وظائف مضافة (Admin – New Functions)
33	4.6.3 مخططات النشاط (Activity Diagrams)
35	4.6.3 مخططات التسلسل (Sequence Diagrams)
38	مخطط تسلسل المدير: إدارة قائمة الأسعار والخدمات (Manage Price List – UC-A-07)
43	4-6-4 مخطط الفئات (Class Diagram)
44	5-6-7 مصفوفة تتبع المتطلبات – الجديدة (Requirements Tracking Matrix)
46	الفصل الخامس: التصميم المعماري للنظام
47	5.1 التصميم عالي المستوى (High-Level Design)
47	5.1.1 مخطط الكتل المعمارية (System Block Diagram)
48	5.2 نمط التصميم المعماري المستخدم
48	5.3 التصميم المعماري للنظام
49	5.4 تصميم قاعدة البيانات
49	5.4.1 نموذج الكيانات والارتباطات (ER Diagram)
50	5.4.2 المخطط العلائقي (Relational Schema)
51	5.5 القيود والمفاتيح (Constraints & Keys)
51	5.6 تصميم الواجهات الخارجية (APIs)
52	الفصل السادس: بيئة التنفيذ والأدوات
53	6.1 الأدوات البرمجية والمكتبات المستخدمة
53	6.2 بيئة التطوير وبنية المشروع
53	6.3 مخدم قاعدة البيانات ونظام التشغيل

53	6.4 المكتبات والتبعيات
54	6.5 أدوات إدارة الإصدارات والسيطرة المصدرية
54	6.6 أدوات التكامل والنشر المستمر (CI/CD)
54	6.7 بيئة الاختبار (Testing Environment)
55	الفصل السابع: التنفيذ البرمجي
56	7.1 مقدمة
56	7.2 هيكل الملفات والمشروع
56	7.3 تنفيذ الوحدات النمطية الرئيسية
56	7.3.1 وحدة إدارة الحسابات والمصادقة
56	7.3.2 وحدة إدارة المواعيد
57	7.3.3 وحدة السجل الطبي
57	7.3.4 وحدة الصيدلية والمخزون
57	4.7 تنفيذ طبقة الوصول إلى البيانات
57	7.5 تنفيذ طبقة الأعمال (Business Logic)
57	7.6 تنفيذ واجهة المستخدم (UI)
58	7.7 الخلاصة
59	الفصل الثامن: الاختبار والتقييم
60	8.1 مقدمة
60	8.2 خطة الاختبار
60	8.3 أنواع الاختبارات المجرأة
60	8.3.1 اختبارات الوحدة (Unit Testing)
60	8.3.2 اختبارات التكامل (Integration Testing)
60	8.3.3 اختبارات النظام (System Testing)
60	8.3.4 اختبارات قابلية الاستخدام (Usability Testing)
61	8.4 نتائج الاختبار
61	8.5 تحليل نتائج الاختبار
61	8.6 مقاييس الجودة

الفصل الأول : المقدمة

1.1 المقدمة

يشهد العالم تطوراً متسارعاً في مجال التحول الرقمي، وأصبحت المؤسسات الصحية من أهم القطاعات التي تعتمد على الأنظمة الإلكترونية لإدارة معلومات المرضى وتنظيم العمل الطبي. وعلى الرغم من هذا التطور، لا تزال العديد من المراكز الصحية تعتمد على السجلات الورقية، مما يؤدي لصعوبات مثل ضياع البيانات وتضارب المواعيد. لذا، يهدف هذا المشروع لتطوير نظام إلكتروني متكامل لأتمتة العمليات بطريقة آمنة وسهلة.

1.2 المراكز الصحية

المركز الصحي هو منشأة تقدم خدمات رعاية أولية تشمل المعاینات الطبية، متابعة الأمراض المزمنة، صرف الأدوية، والتوعية الصحية. وتعتبر هذه المراكز حلقة الوصل الأساسية بين المستفيد والمنشآت الطبية الأكبر كالمستشفيات.

1.3 تحديد المشكلة

تتمثل المشكلة في عدة نقاط رئيسية:

- صعوبة الوصول للسجلات الطبية بسبب الأرشيف الورقي.
- تضارب المواعيد لغياب نظام موحد لإدارة الحجوزات.
- تكرار الأخطاء الطبية والإدارية بسبب عدم ترابط الأقسام.
- ضعف الرقابة على مخزون الأدوية وتاريخ صلاحيتها.

1.4 فكرة المشروع

إنشاء منصة إلكترونية متكاملة لإدارة مركز صحي (صغير أو متوسط)، تشمل وحدات لتسجيل المرضى، حجز المواعيد، السجلات الطبية، الوصفات الإلكترونية، إدارة المخزون، والفوترة.

1.5 هدف المشروع

- أتمتة العمليات بالكامل لتوفير تجربة أفضل للمرضى.
- تحسين كفاءة الأطباء في الوصول للمعلومات وتوثيقها.
- إدارة المواعيد بفعالية ومنع التضارب.
- توفير تنبيهات ذكية للمخزون وفواتير دقيقة.

1.6 الحل المقترح للمشكلة

تطوير نظام إلكتروني موحد ينشئ قاعدة بيانات مركزية تجمع كافة المعلومات (مرضى، سجلات، مواعيد، مخزون). يربط النظام بين أقسام الطبيب والإدارة والصيدلية، ويوفر بوابة إلكترونية للمرضى لمتابعة حالتهم.

1.7 حدود النظام

- ما يشمل النظام: إدارة الحسابات، المواعيد، السجلات الطبية، الوصفات الإلكترونية، الصيدلية، ونظام فوترة أساسي.
- خارج النطاق (الإصدارات المستقبلية): الأنظمة المالية المتقدمة، شركات التأمين، إدارة المختبرات والأشعة، وإدارة المرضى المقيمين.

1.8 الخلاصة

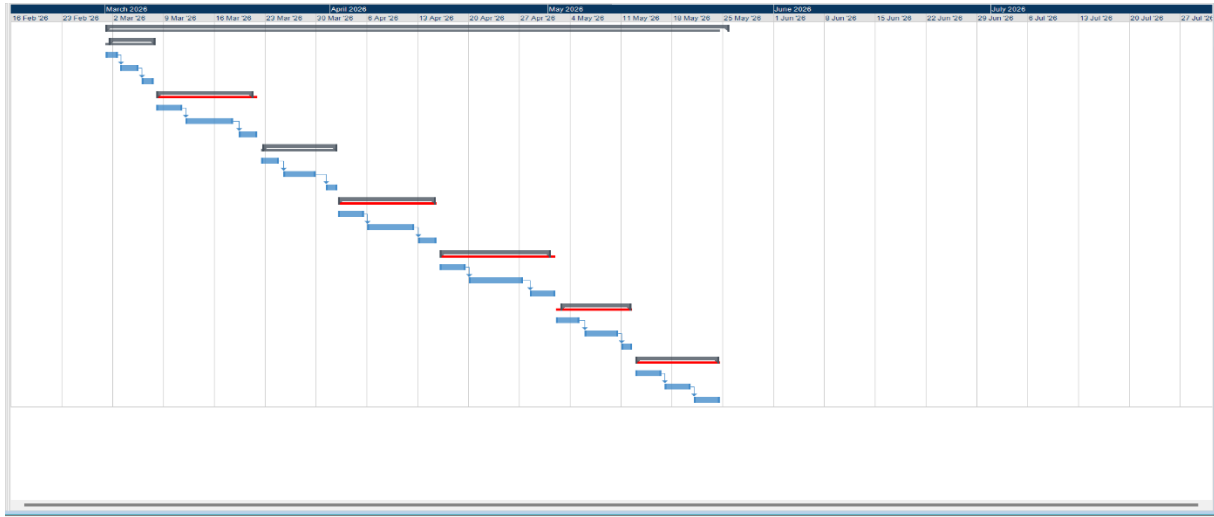
تتضح الحاجة الملحة لتطوير هذا النظام لتعزيز الأداء الطبي والإداري وتوحيد السجلات، مما يشكل أساساً قوياً لبنية صحية رقمية قابلة للتوسع.

الفصل الثاني : المفاهيم الأساسية والدراسة المرجعية

2.1 منهجية التطوير المختارة والمبررات

تم اختيار **المنهجية التزايدية (Incremental Model)** لتطوير نظام HCMS. تكمن مبررات هذا الاختيار في طبيعة الأنظمة الصحية التي تتطلب دقة عالية؛ حيث نتيج لنا هذه المنهجية تقسيم النظام إلى زيادات وظيفية مستقلة (مثل وحدة الاستقبال، ثم عيادة الطبيب، ثم الصيدلية). يساعدنا هذا الأسلوب على تقليل المخاطر من خلال اختبار كل جزء وتدقيق بياناته برمجياً قبل دمج الجزء التالي، مما يضمن استقرار النظام تدريجياً.

2.2 مخطط جانت (Gantt Chart)



	!	Sch...	Task Name	Duration	Start	End	Requirments
1			HCMS	65.65 d...	01/03/2026	25/05/2026	
2			Iteration1: الأساسيات	7 days	01/03/2026	07/03/2026	19 .. 20
3			Analysis & Design	2 days	01/03/2026	02/03/2026	
4			Implementation	3 days	03/03/2026	05/03/2026	
5			Testing	2 days	06/03/2026	07/03/2026	
6			Iteration2: ادارة الاموية والمخزون	14 days	08/03/2026	21/03/2026	21 .. 24
7			Analysis & Design	4 days	08/03/2026	11/03/2026	
8			Implementation	7 days	12/03/2026	18/03/2026	
9			Testing	3 days	19/03/2026	21/03/2026	
10			Iteration3: الفواتير والتقارير المالية للتصديلي	11 days	22/03/2026	01/04/2026	25 .. 26
11			Analysis & Design	3 days	22/03/2026	24/03/2026	
12			Implementation	5 days	25/03/2026	29/03/2026	
13			Testing	2 days	31/03/2026	01/04/2026	
14			Iteration4: ادارة الموردین والتحكم بالاسعار	14 days	02/04/2026	15/04/2026	27 .. 31
15			Analysis & Design	4 days	02/04/2026	05/04/2026	
16			Implementation	7 days	06/04/2026	12/04/2026	
17			Testing	3 days	13/04/2026	15/04/2026	
18			Iteration5: ادارة المواعيد والمرضى	16 days	16/04/2026	01/05/2026	32 .. 36
19			Analysis & Design	4 days	16/04/2026	19/04/2026	
20			Implementation	8 days	20/04/2026	27/04/2026	
21			Testing	4 days	28/04/2026	01/05/2026	
22			Iteration6: تصنيفات المواعيد	10 days	02/05/2026	12/05/2026	37 .. 43
23			Analysis & Design	3 days	02/05/2026	05/05/2026	
24			Implementation	5 days	06/05/2026	10/05/2026	
25			Testing	2 days	11/05/2026	12/05/2026	
26			Iteration7: المحتوى والتواصل	12 days	13/05/2026	24/05/2026	43 .. 46
27			Analysis & Design	4 days	13/05/2026	16/05/2026	
28			Implementation	4 days	17/05/2026	20/05/2026	
29			Testing	4 days	21/05/2026	24/05/2026	

2.3 إدارة المخاطر

تُعد إدارة المخاطر عنصرًا مهمًا لضمان نجاح المشروع واستمراره، حيث يتم من خلالها تحديد المخاطر المحتملة ووضع خطط مناسبة للتعامل معها والحد من آثارها. خلال تطوير هذا المشروع، تم تحديد مجموعة من المخاطر الرئيسية مع آليات معالجتها كما يلي:

الخطر المحتمل	الوصف	آلية المعالجة
تأخر التنفيذ	قد يحدث بسبب سوء تقدير الوقت أو تراكم المهام	إعداد خطة زمنية دقيقة، تقسيم العمل إلى مراحل صغيرة، والمتابعة المستمرة للتقدم
فقدان البيانات	فقدان أو تلف البيانات مما يؤثر على سير المشروع	تنفيذ نسخ احتياطية دورية وتخزينها في أكثر من موقع لضمان الاسترجاع
الأخطاء البرمجية	ظهور أخطاء تؤثر على أداء النظام واستقراره	استخدام الاختبارات المستمرة، اختبارات الوحدات، ومراجعة الكود بشكل دوري
نقص البيانات الحقيقية	عدم توفر بيانات واقعية كافية للاختبار أو التطوير	استخدام بيانات تجريبية تحاكي الواقع لضمان اختبار النظام بشكل شامل
		خلاصة

2.4 فريق العمل وتوزيع المهام والأدوار

تم توزيع الأدوار بناءً على المنهجية المتبعة لضمان توازي العمل في بعض المراحل:

- **أيمن صليبي** إدارة قاعدة البيانات وتطوير منطق العمل (Back-end) لكل زيادة.
- **حسين فخر الدين عبد النبي**: تصميم واجهات المستخدم (Front-end) لضمان تجربة متناسقة عبر جميع الوحدات.
- **عبدالرحمن البردان**: مسؤول عن اختبار كل "Incremental Release" وتوثيق المتطلبات البرمجية.

2.5 إدارة الجودة في المشروع

تم تطبيق أسلوب اختبار بعد كل زيادة برمجية (Increment) لضمان جودة النظام قبل الانتقال إلى المرحلة التالية من التطوير. ويهدف هذا الأسلوب إلى اكتشاف الأخطاء مبكرًا وتحسين استقرار النظام بشكل تدريجي.

2.6 أدوات إدارة المشروع المستخدمة

تم استخدام مجموعة من الأدوات المساعدة في تنظيم وإدارة العمل داخل المشروع، حيث تم اعتماد (Jira و GitHub Projects) لتتبع المهام وتوزيعها بين أعضاء الفريق ومراقبة تقدم العمل. كما تم استخدام (Trello) لتنظيم المهام بشكل بصري وسهل المتابعة، إضافة إلى (Notion) لتوثيق المعلومات والملاحظات الخاصة بالمشروع، و (Google Sheets) لتنظيم الجداول الزمنية وتوثيق البيانات المتعلقة بالتقدم والإنجاز.

2.7 آليات المراقبة والتقييم المستمر

تتم المراقبة عبر تقييم مخرجات كل مرحلة (Increment) ومقارنتها بالمتطلبات المحددة في الفصل الرابع. يتم عقد جلسات مراجعة مع المشرفين د. ماجدة البكور وم. فرح فارس لتقييم مدى نضج النظام في كل مرحلة.

2.8 إدارة تشكيلة البرمجيات SCM

تم اعتماد نظام (Git) لإدارة الإصدارات (Version Control) في المشروع، حيث يتم توثيق جميع التعديلات على الكود البرمجي بشكل منظم يتيح الرجوع إلى أي نسخة سابقة عند الحاجة. كما تم استخدام فروع (Branches) منفصلة لكل ميزة أو تعديل لضمان عدم تعارض التحديثات بين أعضاء الفريق.

الفصل الثالث : الدراسة التحليلية للنظام

3.1 الدراسات السابقة في مجال الأنظمة الصحية الإلكترونية

تم مراجعة عدد من الدراسات والمشاريع السابقة التي تناولت تصميم وتطوير أنظمة إدارة المراكز والمستشفيات الصحية، وفيما يلي عرض موجز لأبرز هذه الدراسات مع تحليل مساهماتها وثغراتها:

عرض موجز لأبرز هذه الدراسات مع تحليل مساهماتها وثغراتها:

3.1.1 نظام إدارة المستشفى الإلكتروني - Ridhab (2018)

قدّم ريدهاب (2018) نظامًا إلكترونيًا لإدارة المستشفيات يعتمد على الويب، ويشمل وحدات لإدارة المرضى والمواعيد والفوترة، مُطَوَّرًا باستخدام تقنيات PHP وMySQL. وقد أسهم هذا النظام في تحسين سير العمل الإداري داخل المستشفيات، غير أنه افتقر إلى وحدة متكاملة لإدارة الصيدلية والمخزون الدوائي، كما لم يتضمن نظامًا للسجلات الطبية الإلكترونية. علاوة على ذلك، لم يُطبّق فيه نظام صلاحيات متعدد الأدوار بالكفاءة الكافية.

3.1.2 نظام إدارة المستشفى المبني على الويب - Rahman (2022)

ركّز رحمن (2022) على تطوير نظام لإدارة المستشفى يُعالج مشكلة إدارة البيانات الطبية بشكل رقمي، مع دعم متعدد الأدوار (مريض، طبيب، مدير)، مستخدمًا تقنيات ASP.NET وSQL Server. وقد تميّز هذا النظام بواجهة مستخدم منظمة وإدارة جيدة لصلاحيات المستخدمين، إلا أنه لم يشمل إدارة الصيدلية أو المخزون الدوائي، وغابت عنه ميزة التنبيهات الآنية المرتبطة بالمواعيد أو انتهاء صلاحية الأدوية.

3.1.3 نظام إدارة المستشفى - Tuhin (2023)

اقترح توهين (2023) نظامًا شاملاً يهدف إلى رقمنة جميع جوانب إدارة المستشفى، من حجز المواعيد إلى إدارة السجلات الطبية، وذلك باستخدام تقنيات MySQL وSpring Boot Java. وقد امتاز هذا النظام بتغطية وظيفية واسعة، لكنه اعتمد على تقنيات أكثر تعقيدًا مما أثر على إمكانية نشره في البيئات ذات الموارد المحدودة، فضلًا عن ثقل واجهة المستخدم التي لم تُصمّم بما يكفي لتناسب سياقات المراكز الصحية الصغيرة.

3.2 التحليل المقارن للأنظمة الحالية

يوضح الجدول التالي مقارنة بين الأنظمة الصحية المراجعة ونظامنا المقترح (HCMS) من حيث الميزات الوظيفية الرئيسية:

الميزة	Ridhab (2018)	Rahman (2022)	Tuhin (2023)	نظامنا HCMS
إدارة المرضى	✓	✓	✓	✓
إدارة المواعيد	✓	✓	✓	✓
السجل الطبي الإلكتروني	X	جزئي	✓	✓
إدارة الصيدلية والمخزون	X	X	جزئي	✓
نظام الفوترة	✓	X	X	✓
صلاحيات متعددة الأدوار	جزئي	✓	✓	✓
تنبيهات انتهاء الصلاحية	X	X	X	✓
التقنية المستخدمة	PHP/MySQL	ASP.NET/SQL	Java/MySQL	ASP.NET MVC/SQL
سهولة الاستخدام	متوسطة	جيدة	متوسطة	عالية

3.3 الفجوة البحثية والابتكار المقترح

بناءً على مراجعة الأنظمة السابقة وتحليلها المقارن، تبين أن معظم الأنظمة المتاحة تفتقر إلى التكامل الكامل بين جميع وحدات إدارة المركز الصحي ضمن منصة واحدة متناسقة. فعلى الرغم من أن بعض الأنظمة تتميز في مجالات محددة، إلا أن لا نظام منها يجمع في آنٍ واحد بين:

- إدارة متكاملة للمرضى والمواعيد والسجلات الطبية الإلكترونية
- نظام صيدلية شامل مع تتبع المخزون والتنبيه لانتهاء صلاحية الأدوية

- وحدة فوتر مرتبطة بالخدمات الطبية المقدمة
 - نظام صلاحيات متعدد الأدوار (مريض، طبيب، مدير) بواجهة مستخدم سهلة وواضحة
- وهنا تكمن إضافة مشروع HCMS، إذ يسعى إلى سدّ هذه الفجوة من خلال تطوير نظام متكامل يجمع جميع هذه العناصر ضمن منصة ويب واحدة مبنية بتقنيات ASP.NET MVC و SQL Server، مما يجعله حلاً عملياً شاملاً ومناسباً للمراكز الصحية الصغيرة والمتوسطة.

3.4 التقنيات والأطر الحديثة في المجال

يعتمد مشروع HCMS على مجموعة من التقنيات الحديثة التي أثبتت كفاءتها في تطوير تطبيقات الويب:

ASP.NET MVC

إطار عمل متقدم من مايكروسوفت يُستخدم لبناء تطبيقات ويب منظمة وفق نمط Model-View-Controller، مما يضمن فصل المسؤوليات وسهولة الصيانة والتطوير. يتميز بدعم قوي للأمان وإدارة الجلسات والصلاحيات.

SQL Server

نظام إدارة قواعد بيانات علائقية من مايكروسوفت يُستخدم لتخزين البيانات الطبية والإدارية بأمان وكفاءة عالية، مع دعم للمعاملات المتزامنة والنسخ الاحتياطية التلقائية.

Bootstrap

إطار عمل للواجهة الأمامية يُسهّل تصميم واجهات مستخدم متجاوبة تعمل على مختلف أحجام الشاشات، مما يضمن تجربة مستخدم سلسة لكل من يستخدم النظام سواء على الحاسوب أو الهاتف.

JavaScript و HTML/CSS

تقنيات الويب الأساسية التي تُستخدم لتحسين التفاعل مع المستخدم، والتحقق من صحة المدخلات، وعرض البيانات بشكل ديناميكي دون إعادة تحميل الصفحة.

Git و GitHub

أدوات التحكم بالإصدارات المستخدمة لإدارة الكود المصدري وضمان التعاون الفعال بين أعضاء الفريق، مع إمكانية تتبع جميع التغييرات والعودة إلى أي نسخة سابقة.

3.5 الإطار النظري والدراسات المرجعية

يستند مشروع HCMS إلى عدد من المفاهيم والنظريات الهندسية والحاسوبية التي تشكل الأساس النظري للنظام:

نمط MVC (Model-View-Controller)

يُعدّ نمط MVC الركيزة المعمارية الأساسية للنظام، وهو نمط تصميمي معماري يقوم على فصل النظام إلى ثلاث طبقات متميزة: طبقة النموذج (Model) المسؤولة عن منطق الأعمال والبيانات، وطبقة العرض (View) المسؤولة عن واجهة المستخدم، والمتحكم (Controller) الذي يُنسّق بينهما. هذا الفصل يُسهّل الصيانة والتطوير المستقبلي.

نظرية التحكم بالوصول المبني على الأدوار (RBAC)

يعتمد النظام على مبدأ Role-Based Access Control لضمان أن كل مستخدم (مريض، طبيب، مدير) يصل فقط إلى الوظائف المخصصة له، مما يعزز أمن البيانات ويحمي خصوصية المرضى.

النمذجة العلائقية للبيانات (Relational Data Modeling)

تستند بنية قاعدة البيانات إلى مبادئ النمذجة العلائقية وقواعد التطبيع (Normalization) لضمان تجنب التكرار وتحقيق تكامل البيانات، مع توظيف المفاتيح الأساسية والخارجية لبناء علاقات منطقية بين الجداول.

3.6 التوجهات الحديثة والمستقبلية في المجال

يشهد مجال أنظمة إدارة المراكز الصحية تطورات متسارعة، ومن أبرز التوجهات الحديثة والمستقبلية التي يمكن أن يستفيد منها نظام HCMS مستقبلاً:

- الذكاء الاصطناعي وتعلم الآلة (AI/ML): توظيف نماذج التنبؤ لاقتراح التشخيصات، وتحليل بيانات المرضى لاكتشاف الأنماط المرضية المبكرة.
- تحليلات البيانات الضخمة (Data Analytics): استخدام تقنيات تحليل البيانات لمساعدة الإدارة في اتخاذ قرارات مبنية على الإحصاءات الطبية والتشغيلية.
- تطبيقات الهواتف المحمولة (Mobile Apps): تطوير تطبيقات جوال تتيح للمرضى حجز المواعيد ومتابعة وصفاتهم الطبية عن بُعد.
- التكامل مع الأنظمة الوطنية: ربط النظام مع قواعد البيانات الصحية الوطنية لتبادل المعلومات وتحسين متابعة الحالات الصحية على مستوى أوسع.
- أمن المعلومات المتقدم: تطبيق تقنيات المصادقة الثنائية وتشفير البيانات الحساسة لرفع مستوى الحماية.

الفصل الرابع: نمذجة وتحليل المتطلبات

4.1 دراسة الجدوى

4.1.1 الجدوى الفنية

تتوفر جميع التقنيات اللازمة لتطوير النظام، بما يشمل إطار عمل ASP.NET MVC المدعوم من مايكروسوفت، وقاعدة بيانات SQL Server، وبيئة التطوير Visual Studio. كما تتوفر المكتبات والحزم اللازمة عبر NuGet Package Manager، مما يضمن إمكانية بناء النظام بكفاءة ضمن الإمكانيات المتاحة.

4.1.2 الجدوى التشغيلية

يُعالج النظام مشكلة حقيقية تواجهها المراكز الصحية، ويوفر واجهات مستخدم سهلة تناسب المستخدمين من مستويات مختلفة. ومن المتوقع أن يُقلّل النظام من الاعتماد على الإجراءات الورقية بنسبة لا تقل عن 80%، ويُحسّن من كفاءة إدارة المواعيد والسجلات الطبية.

4.1.3 الجدوى الاقتصادية

يعتمد المشروع على تقنيات مفتوحة المصدر أو ذات تراخيص تعليمية متاحة، مما يُقلّل من تكاليف التطوير. وتتضمن التكاليف الرئيسية بيئة التطوير وخادم الاستضافة المحلي (localhost)، وهي تكاليف محدودة نظرًا لطبيعة المشروع الأكاديمية.

4.2 تحديد أصحاب المصلحة وتحليل احتياجاتهم

أصحاب المصلحة	دورهم	احتياجاتهم الأساسية	توقعاتهم
المريض	المستخدم النهائي الأساسي	حجز مواعيد بسهولة، عرض السجل الطبي والوصفات، إدارة الملف الشخصي	واجهة بسيطة وسهلة الاستخدام، الخصوصية والأمان
الطبيب	مستخدم متخصص	الاطلاع على مواعيده، إدارة السجلات الطبية، كتابة الوصفات	سرعة الوصول للمعلومات، موثوقية البيانات
مدير النظام	مستخدم إداري	إدارة شاملة للمواعيد والمخزون والحسابات	صلاحيات كاملة، تقارير دقيقة، تنبيهات فورية
المطور المستقبلي	مستفيد غير مباشر	كود منظم قابل للتطوير والتوسع	توثيق جيد، هيكلية واضحة، معايير برمجية موحدة

4.3 المتطلبات الوظيفية (Functional Requirements)

م	رقم الحالة	اسم المتطلب الوظيفي	الفاعل	الفئة
1	UC-Ph-01	تسجيل الدخول (Log In)	الصيدلي	الصيدلي
2	UC-Ph-02	عرض تفاصيل وصفة طبية	الصيدلي	الصيدلي
3	UC-Ph-03	إضافة دواء جديد (Add New Medicine)	الصيدلي	الصيدلي
4	UC-Ph-04	تعديل دواء (Update Medicine)	الصيدلي	الصيدلي
5	UC-Ph-05	حذف دواء (Delete Medicine)	الصيدلي	الصيدلي
6	UC-Ph-06	إدارة تنبيهات انتهاء الصلاحية	الصيدلي	الصيدلي
7	UC-Ph-07	إنشاء التقرير اليومي بالفواتير الصادرة للمدير	الصيدلي	الصيدلي
8	UC-Ph-08	تسجيل طلب شراء أدوية	الصيدلي	الصيدلي
9	UC-A-06	إضافة مورد جديد	المدير	المدير
10	UC-A-07	حذف مورد	المدير	المدير
11	UC-A-08	عرض قائمة الموردين	المدير	المدير
12	UC-A-09	التحكم بلائحة الأسعار والخدمات	المدير	المدير
13	UC-A-10	التعامل مع التقارير اليومية من العيادات والمخزون	المدير	المدير
14	UC-A-11	تعطيل حساب مريض	المدير	المدير
15	UC-A-12	تسجيل طلب إجازة لطبيب	المدير	المدير
16	UC-D-08	طباعة تقرير يومي بالمرضى الزائرين	الطبيب	الطبيب
17	UC-D-09	تقديم طلب إجازة	الطبيب	الطبيب
18	UC-AI-01	عرض مؤشر خطر عدم الحضور	مدير، طبيب	الذكاء الاصطناعي
19	UC-AI-02	تصفية المواعيد حسب مستوى الخطر	مدير، طبيب	الذكاء الاصطناعي

4.4 المتطلبات غير الوظيفية (Non-Functional Requirements)

المتطلب	الفئة
زمن تنفيذ العمليات الأساسية (تسجيل دخول، حجز موعد) يجب ألا يتجاوز 3 ثوانٍ	الأداء
زمن استرجاع بيانات المريض يجب ألا يتجاوز 2 ثانية	الأداء
تطبيق نظام التحكم بالوصول المبني على الأدوار (RBAC) لضمان الفصل بين صلاحيات المستخدمين	الأمان
تشفير كلمات المرور باستخدام خوارزميات التشفير الآمنة قبل تخزينها	الأمان
يجب أن تكون واجهة المستخدم بسيطة وقابلة للاستخدام دون تدريب مسبق	قابلية الاستخدام
يجب أن يكون النظام متاحًا بنسبة لا تقل عن 99% من وقت التشغيل	الموثوقية
يجب أن يكون الكود البرمجي موثقًا ومنظمًا وفق نمط MVC	قابلية الصيانة
يجب أن يعمل النظام بكفاءة على متصفحات Chrome و Firefox و Edge الحديثة	التوافق

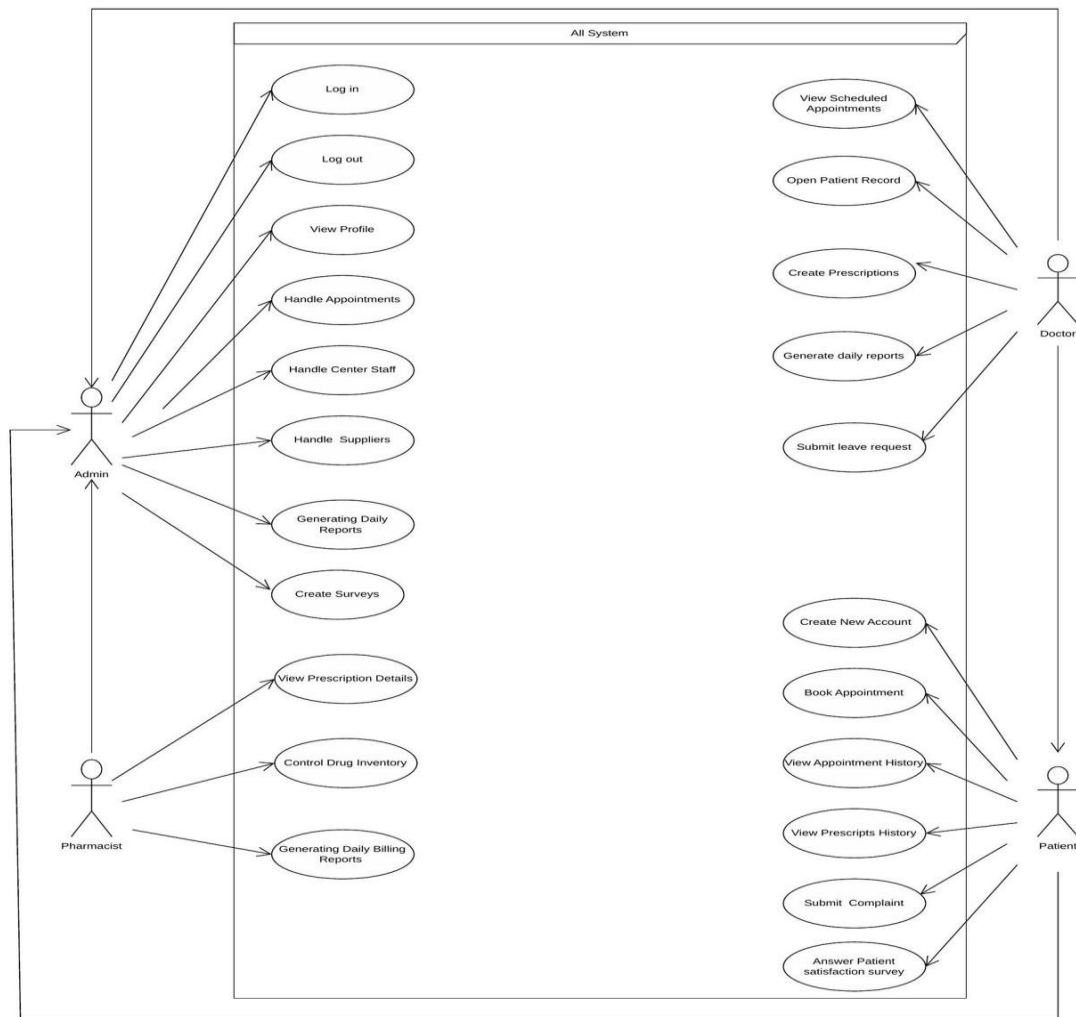
4.5 قائمة أولويات المتطلبات (MoSCoW)

المتطلبات	الأولوية
تسجيل الدخول والخروج، إدارة المواعيد، السجل الطبي الإلكتروني، إدارة الصيدلية، نظام الفوترة، صلاحيات متعددة الأدوار	Must Have (يجب تنفيذه)
التنبيهات الآنية لانتهاج صلاحية الأدوية، إضافة الملاحظات السريرية، تقارير إحصائية أساسية	Should Have (يفضل تنفيذه)
لوحة تحكم إحصائية متقدمة، إمكانية طباعة الوصفات الطبية، دعم اللغات المتعددة	Could Have (يمكن تنفيذه)
تطبيق الهاتف المحمول، التكامل مع أنظمة التأمين الصحي، الذكاء الاصطناعي التنبؤي	Won't Have (خارج النطاق حالياً)

4.6 مخططات UML للمتطلبات

4.6.1 حالات الاستخدام الرئيسية (Use Cases)

يوضح مخطط حالات الاستخدام الكامل التالي تفاعل جميع المستخدمين (المريض، الطبيب، المدير، الصيدلي) مع النظام وحالات الاستخدام الخاصة بكل منهم:



مخطط حالات الاستخدام 1: النظام الكامل (All System Use Case)

4.6.2 توصيف حالات الاستخدام (Use Case Descriptions)

يلي توصيف حالات الاستخدام الجديدة المضافة في هذا الإصدار للصيدي والمدير (المتطلبات REQ01-REQ13 الجديدة):

أولاً: الصيدلي (Pharmacist)

جدول توصيف الصيدلي 1.4 : تسجيل الدخول

اسم الحالة	تسجيل الدخول (Log In)
رقم الحالة	UC-Ph-01
الفاعلون	الصيدلي
الشروط السابقة	يجب أن يكون للصيدلي حساب مسجل في النظام
الشروط اللاحقة	يحصل الصيدلي على لوحة التحكم الخاصة به
تسلسل الأحداث	1- يطلب الصيدلي تسجيل الدخول 2- يعرض النظام نموذج تسجيل الدخول 3- يدخل الصيدلي البريد الإلكتروني وكلمة المرور 4- يتحقق النظام من صحة البيانات 5- يمنح النظام الصيدلي الوصول إلى لوحة التحكم
تسلسل بديل	إذا كانت بيانات الدخول غير صحيحة يعرض النظام رسالة خطأ ويبقى النموذج مفتوحاً
الاستثناءات	محاولات تسجيل دخول متكررة فاشلة

جدول توصيف الصيدلي 2.4 : عرض تفاصيل الوصفات

اسم الحالة	عرض تفاصيل الوصفات (View Prescription Details)
رقم الحالة	UC-Ph-02
الفاعلون	الصيدلي
الشروط السابقة	يجب أن يكون الصيدلي مسجل الدخول
الشروط اللاحقة	يعرض النظام تفاصيل الوصفة الطبية
تسلسل الأحداث	1- يدخل الصيدلي إلى قائمة الوصفات المعلقة 2- يختار الوصفة المطلوبة 3- يعرض النظام الأدوية والجرعات والمريض والطبيب
تسلسل بديل	إذا لم توجد وصفات معلقة يعرض النظام رسالة لا توجد وصفات
الاستثناءات	خطأ في استرجاع بيانات الوصفة

جدول توصيف الصيدلي 3.4 : إضافة دواء جديد

اسم الحالة	إضافة دواء جديد (Add New Medicine)
رقم الحالة	UC-Ph-03
الفاعلون	الصيدلي
الشروط السابقة	يجب أن يكون الصيدلي مسجل الدخول وله صلاحية الوصول للمستودع
الشروط اللاحقة	يتم تحديث قاعدة بيانات الأدوية وتظهر الكمية الجديدة في المخزون

تسلسل الأحداث	1- يطلب الصيدلي إضافة دواء جديد 2- يعرض النظام نموذج إدخال البيانات 3- يدخل الصيدلي بيانات الدواء (الاسم، الشركة، تاريخ انتهاء الصلاحية، الكمية، السعر) 4- يتحقق النظام من اكتمال البيانات 5- يضغط الصيدلي حفظ 6- يخزن النظام الدواء ويصدر تأكيداً
تسلسل بديل	إذا كان الدواء موجوداً مسبقاً يقترح النظام تحديث الكمية
الاستثناءات	إدخال تاريخ انتهاء صلاحية قديم أو ترك حقول إجبارية فارغة

جدول توصيف الصيدلي 4.4 : تعديل دواء

اسم الحالة	تعديل دواء (Update Medicine)
رقم الحالة	UC-Ph-04
الفاعلون	الصيدلي
الشروط السابقة	الدواء يجب أن يكون موجوداً مسبقاً في قاعدة البيانات
الشروط اللاحقة	تحديث بيانات الدواء في النظام
تسلسل الأحداث	1- يدخل الصيدلي إلى قائمة الأدوية 2- يبحث عن الدواء المطلوب 3- يختار خيار تعديل 4- يعدل الحقول المطلوبة 5- يضغط تحديث 6- يؤكد النظام نجاح التعديل
تسلسل بديل	لا يوجد
الاستثناءات	عدم العثور على الدواء أو إدخال قيم غير منطقية

جدول توصيف الصيدلي 5.4 : حذف دواء

اسم الحالة	حذف دواء (Delete Medicine)
رقم الحالة	UC-Ph-05
الفاعلون	الصيدلي
الشروط السابقة	الدواء موجود في النظام وليس له عمليات صرف معلقة
الشروط اللاحقة	إزالة الدواء من القائمة النشطة
تسلسل الأحداث	1- يختار الصيدلي الدواء المراد حذفه 2- يضغط حذف 3- يعرض النظام رسالة تأكيد 4- يؤكد الصيدلي العملية 5- يحذف النظام السجل ويصدر تأكيداً
تسلسل بديل	إلغاء عملية الحذف والعودة للقائمة
الاستثناءات	منع الحذف إذا كان الدواء مرتبطاً بوصفات حالية

جدول توصيف الصيدلي 6.4 : إدارة تنبيهات انتهاء الصلاحية

اسم الحالة	إدارة تنبيهات انتهاء الصلاحية (Manage Medication Expiry Alerts)
رقم الحالة	UC-Ph-06
الفاعلون	الصيدلي
الشروط السابقة	وجود أدوية في المخزون بتواريخ صلاحية مسجلة
الشروط اللاحقة	تأمين سلامة المرضى عبر عزل الأدوية المنتهية أو القريبة من الانتهاء

تسلسل الأحداث	1- يفحص النظام قاعدة بيانات المخزون يومياً تلقائياً 2- يحدد الأدوية التي ستنتهي صلاحيتها خلال 30 يوماً 3- يظهر تنبيه في لوحة تحكم الصيدلي 4- يراجع الصيدلي القائمة ويتخذ قراراً (إتلاف أو إرجاع للمورد) 5- يحدث الصيدلي حالة الأدوية في المخزون
تسلسل بديل	ضبط إعدادات التنبيه (تغيير المدة الزمنية)
الاستثناءات	فشل النظام في إرسال التنبيه بسبب خطأ في الخادم

جدول توصيف الصيدلي 7.4 : إنشاء تقرير فواتير يومي للمدير

اسم الحالة	إنشاء تقرير فواتير يومي للمدير (Generate Daily Invoice Report)
رقم الحالة	UC-Ph-07
الفاعلون	الصيدلي
الشروط السابقة	يجب أن يكون الصيدلي مسجل الدخول
الشروط اللاحقة	يُولد تقرير الفواتير اليومي ويصبح متاحاً للمدير
تسلسل الأحداث	1- يطلب الصيدلي إنشاء تقرير يومي 2- يحدد التاريخ المطلوب 3- يجمع النظام جميع الفواتير المرتبطة بذلك اليوم 4- يعرض ملخصاً بإجمالي المبيعات والأدوية المصروفة 5- يُرسل التقرير إلى المدير
تسلسل بديل	إذا لم توجد فواتير لذلك اليوم يعرض النظام تقريراً فارغاً
الاستثناءات	خطأ في استرجاع بيانات الفواتير

جدول توصيف الصيدلي 8.4 : تسجيل طلب شراء دواء

اسم الحالة	تسجيل طلب شراء دواء (Register Medicine Purchase Order)
رقم الحالة	UC-Ph-08
الفاعلون	الصيدلي
الشروط السابقة	يجب أن يكون الصيدلي مسجل الدخول
الشروط اللاحقة	يُسجَل طلب الشراء في النظام ويُرسَل للمدير للاعتماد
تسلسل الأحداث	1- يطلب الصيدلي إنشاء طلب شراء جديد 2- يختار الدواء والمورد والكمية المطلوبة والسعر 3- يراجع بيانات الطلب 4- يؤكد الإرسال 5- يسجل النظام الطلب ويُعلم المدير
تسلسل بديل	إذا لم يكن المورد مسجلاً يطلب النظام إضافته أولاً
الاستثناءات	خطأ في حفظ الطلب أو انقطاع الشبكة

ثانياً: المدير – وظائف مضافة (Admin – New Functions)

جدول توصيف المدير 10.3 : إضافة مورد جديد

اسم الحالة	إضافة مورد جديد (Add New Supplier)
رقم الحالة	UC-A-04
الفاعلون	المدير
الشروط السابقة	يجب أن يكون المدير مسجل الدخول

الفصل الرابع

نمذجة وتحليل المتطلبات

الشروط اللاحقة	يُضاف المورد الجديد إلى قاعدة البيانات
تسلسل الأحداث	1- يختار المدير إضافة مورد جديد 2- يدخل بيانات المورد (الاسم، البريد الإلكتروني، التفاصيل) 3- يتحقق النظام من اكتمال البيانات 4- يحفظ المدير السجل 5- يؤكد النظام نجاح العملية
تسلسل بديل	إذا كان المورد موجوداً مسبقاً يعرض النظام رسالة خطأ
الاستثناءات	ترك حقول إجبارية فارغة

جدول توصيف المدير 11.3 : حذف مورد

اسم الحالة	حذف مورد (Delete Supplier)
رقم الحالة	UC-A-05
الفاعلون	المدير
الشروط السابقة	المورد موجود في النظام
الشروط اللاحقة	إزالة المورد من قاعدة البيانات
تسلسل الأحداث	1- يختار المدير المورد المراد حذفه 2- يضغط حذف 3- يعرض النظام رسالة تأكيد 4- يؤكد المدير 5- يحذف النظام السجل
تسلسل بديل	إلغاء الحذف والعودة للقائمة
الاستثناءات	منع الحذف إذا كان المورد مرتبطاً بطلبات شراء نشطة

جدول توصيف المدير 12.3 : عرض قائمة الموردين

اسم الحالة	عرض قائمة الموردين (View Suppliers List)
رقم الحالة	UC-A-06
الفاعلون	المدير
الشروط السابقة	يجب أن يكون المدير مسجل الدخول
الشروط اللاحقة	يعرض النظام قائمة الموردين المسجلين
تسلسل الأحداث	1- يدخل المدير إلى صفحة إدارة الموردين 2- يسترجع النظام قائمة الموردين 3- يعرض النظام القائمة مع بيانات كل مورد
تسلسل بديل	إذا لم يوجد موردون يعرض النظام رسالة القائمة فارغة
الاستثناءات	خطأ في استرجاع البيانات

جدول توصيف المدير 13.3 : إدارة قائمة الأسعار والخدمات

اسم الحالة	إدارة قائمة الأسعار والخدمات (Manage Price List and Services)
رقم الحالة	UC-A-07
الفاعلون	المدير
الشروط السابقة	يجب أن يكون المدير مسجل الدخول
الشروط اللاحقة	تحديث قائمة الأسعار في النظام

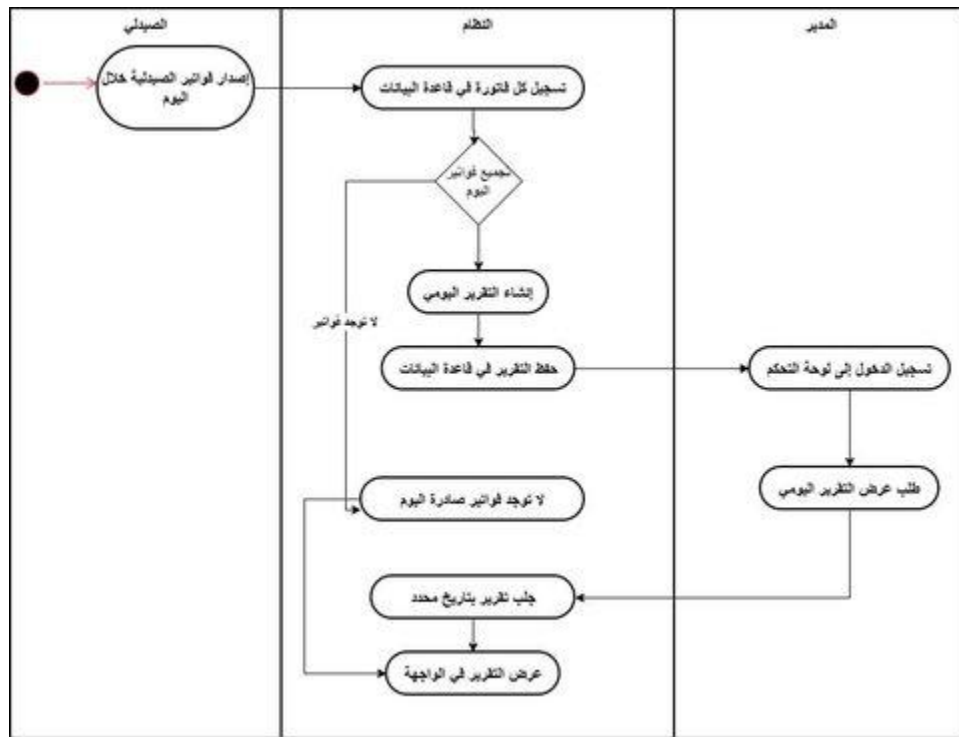
تسلسل الأحداث	1- يدخل المدير إلى صفحة إدارة الأسعار 2- يختار الخدمة أو الدواء المراد تعديل سعره 3- يدخل السعر الجديد 4- يحفظ التعديلات 5- يؤكد النظام التحديث
تسلسل بديل	إضافة خدمة جديدة غير مسجلة
الاستثناءات	إدخال سعر سالب أو قيم غير منطقية

جدول توصيف المدير 14.3 : معالجة التقارير اليومية

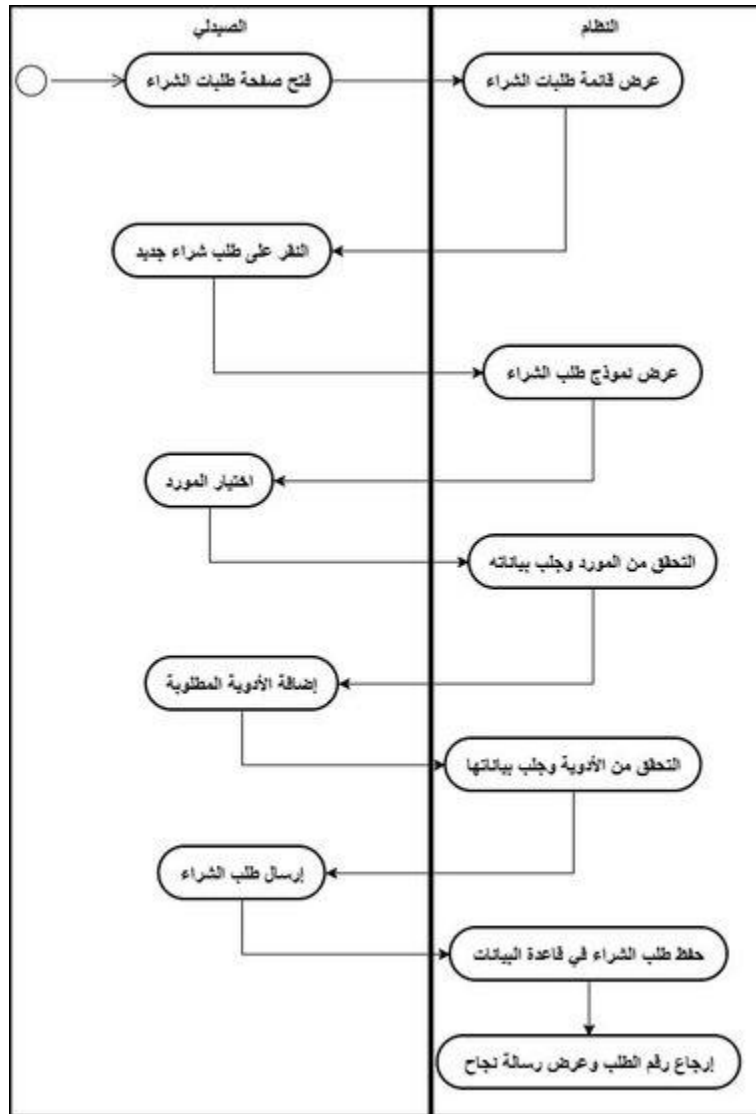
اسم الحالة	معالجة التقارير اليومية (Handle Daily Reports from Clinics & Inventory)
رقم الحالة	UC-A-08
الفاعلون	المدير
الشروط السابقة	يجب أن يكون المدير مسجل الدخول
الشروط اللاحقة	يطلع المدير على التقارير اليومية الشاملة
تسلسل الأحداث	1- يدخل المدير إلى صفحة التقارير 2- يختار التاريخ المطلوب 3- يسترجع النظام بيانات العيادات والمخزون 4- يعرض النظام تقريراً شاملاً
تسلسل بديل	تصفية التقارير حسب العيادة أو القسم
الاستثناءات	خطأ في توليد التقرير

4.6.3 مخططات النشاط (Activity Diagrams)

تُوضّح مخططات النشاط التالية تدفق العمليات الرئيسية في النظام. تشمل مخططات الصيدلي الجديدة:

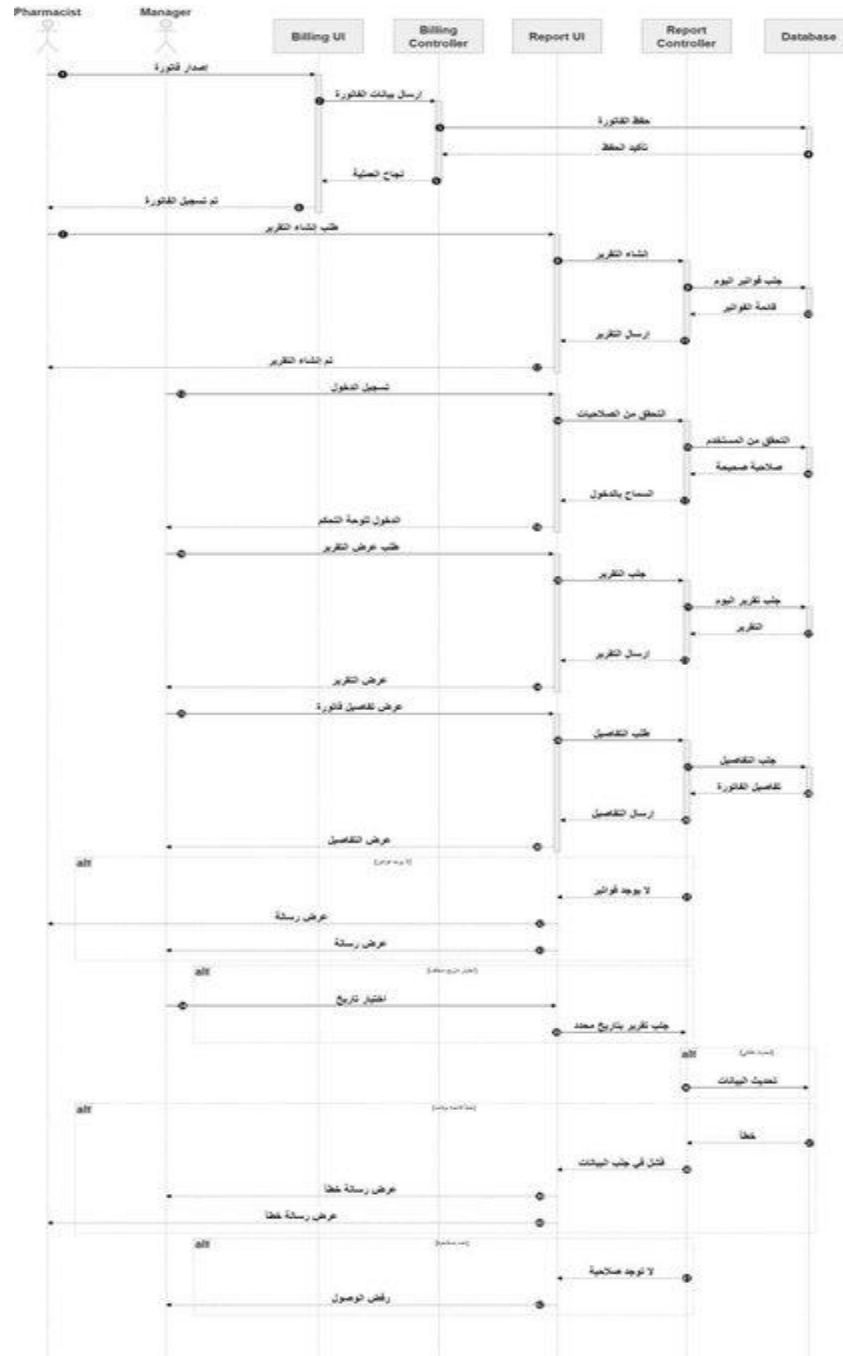


مخطط نشاط الصيدلي 7: إنشاء تقرير فواتير يومي للمدير (Generate Daily Invoice Report)

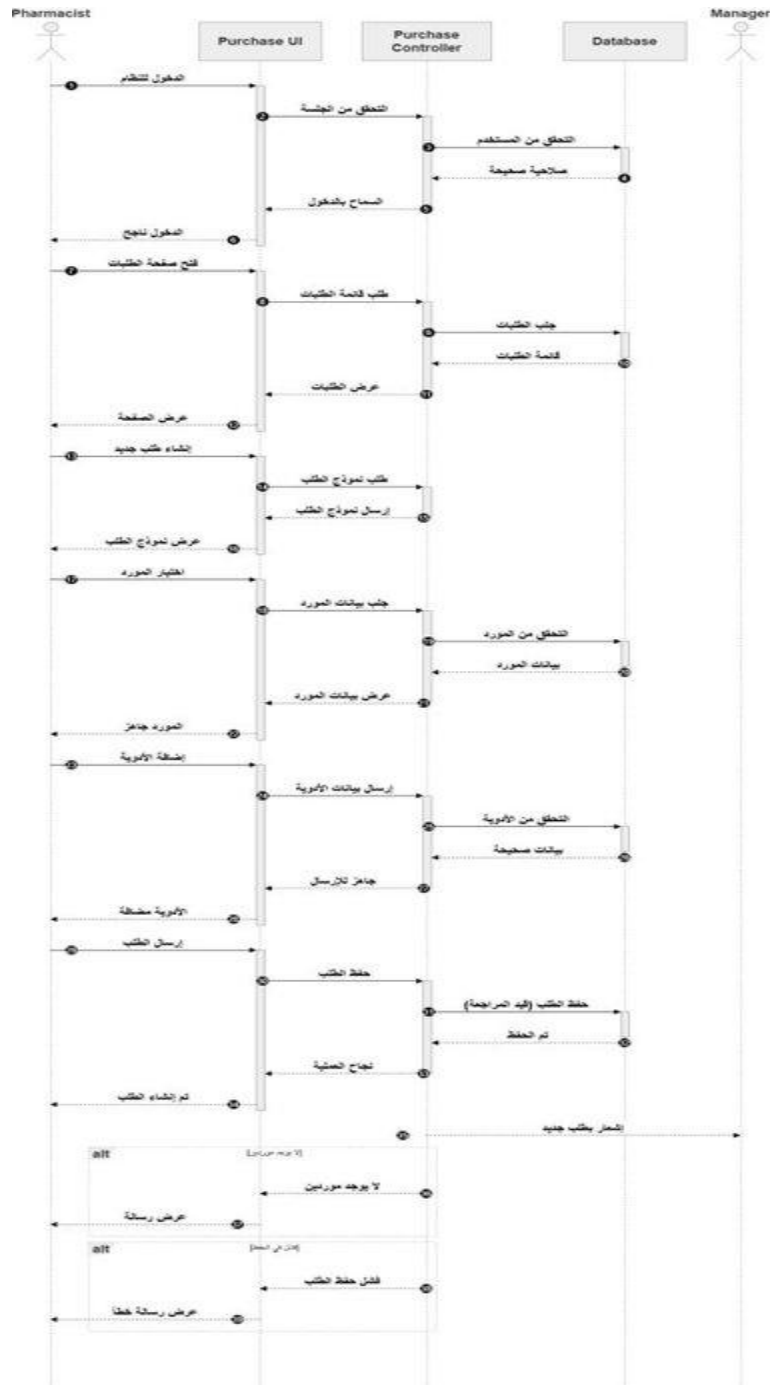


مخطط نشاط الصيدلي 8: تسجيل طلب شراء دواء (Register Medicine Purchase Order)

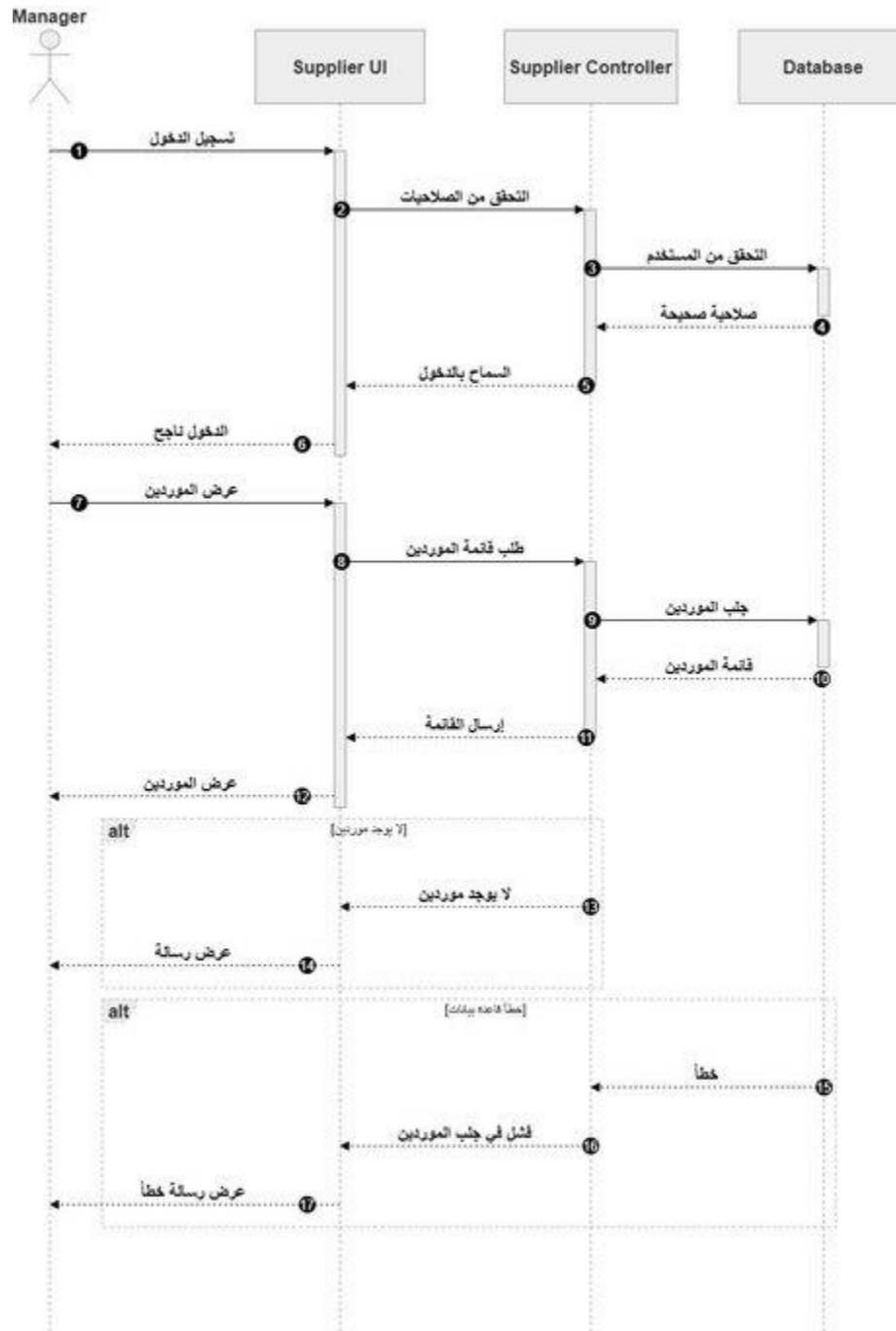
4.6.3 مخططات التسلسل (Sequence Diagrams)



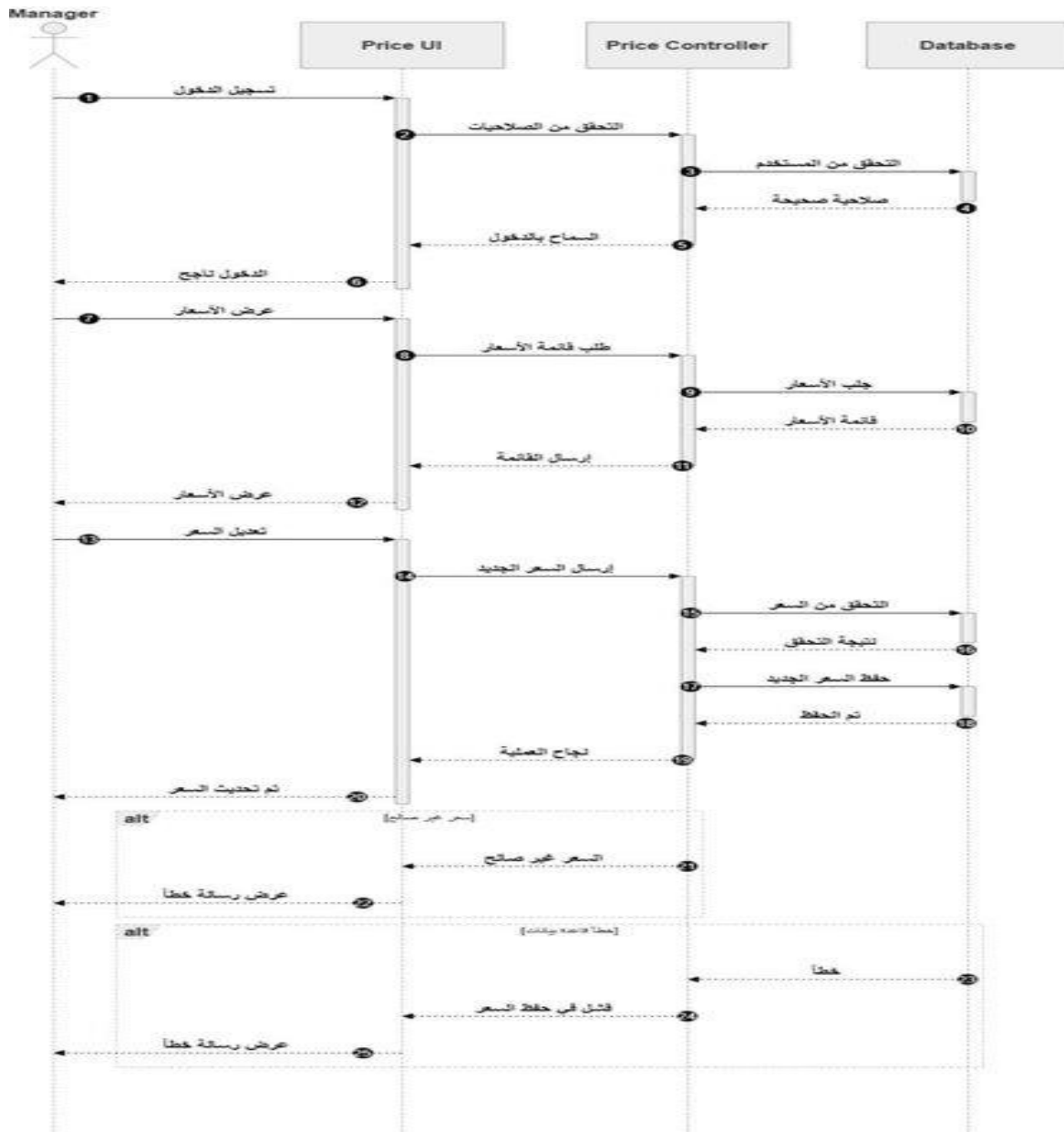
مخطط تسلسل الصيدلي 7: إنشاء تقرير فواتير يومي للمدير (Generate Daily Invoice Report)



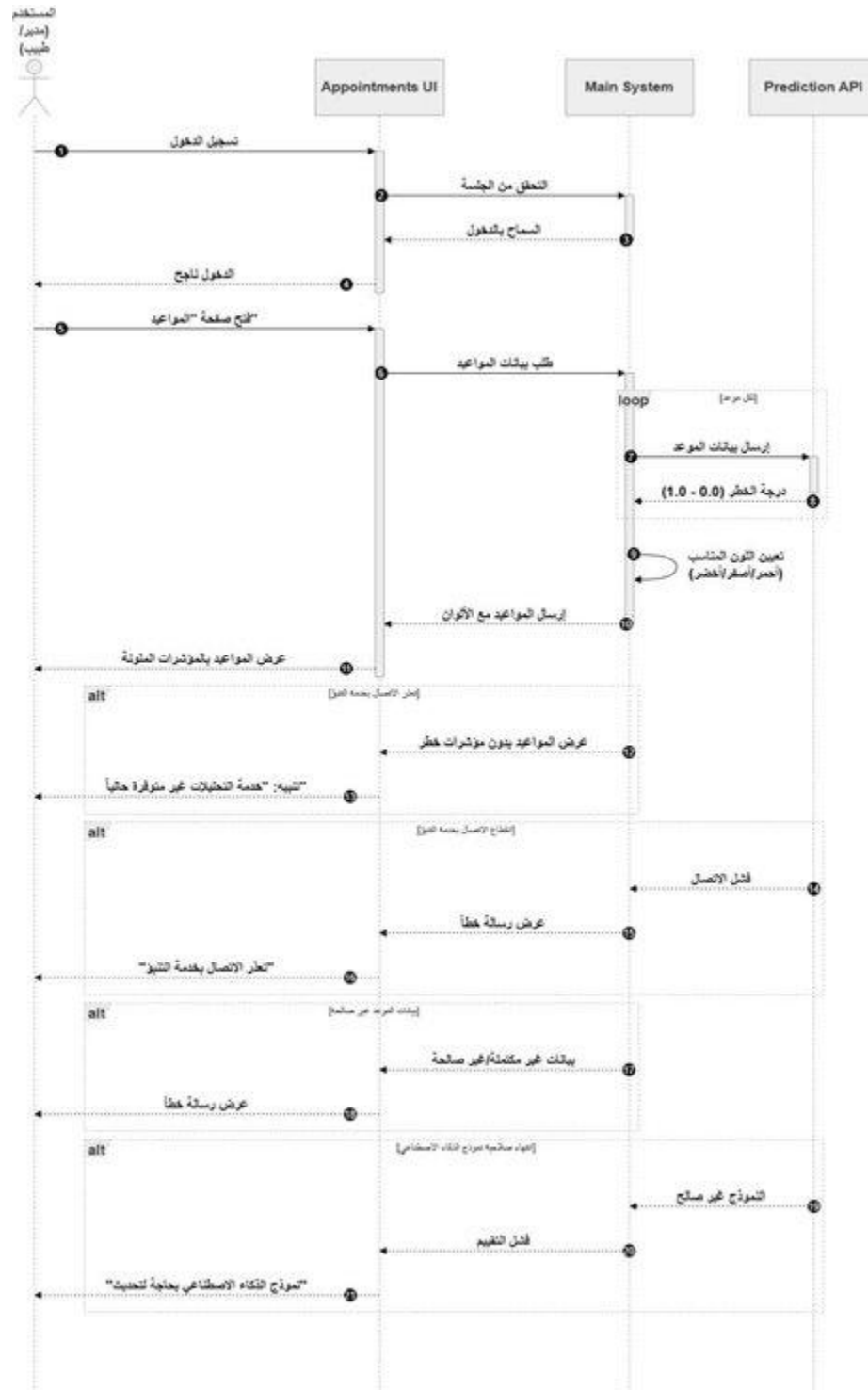
مخطط تسلسل الصيدلي 8: تسجيل طلب شراء دواء (Register Medicine Purchase Order)



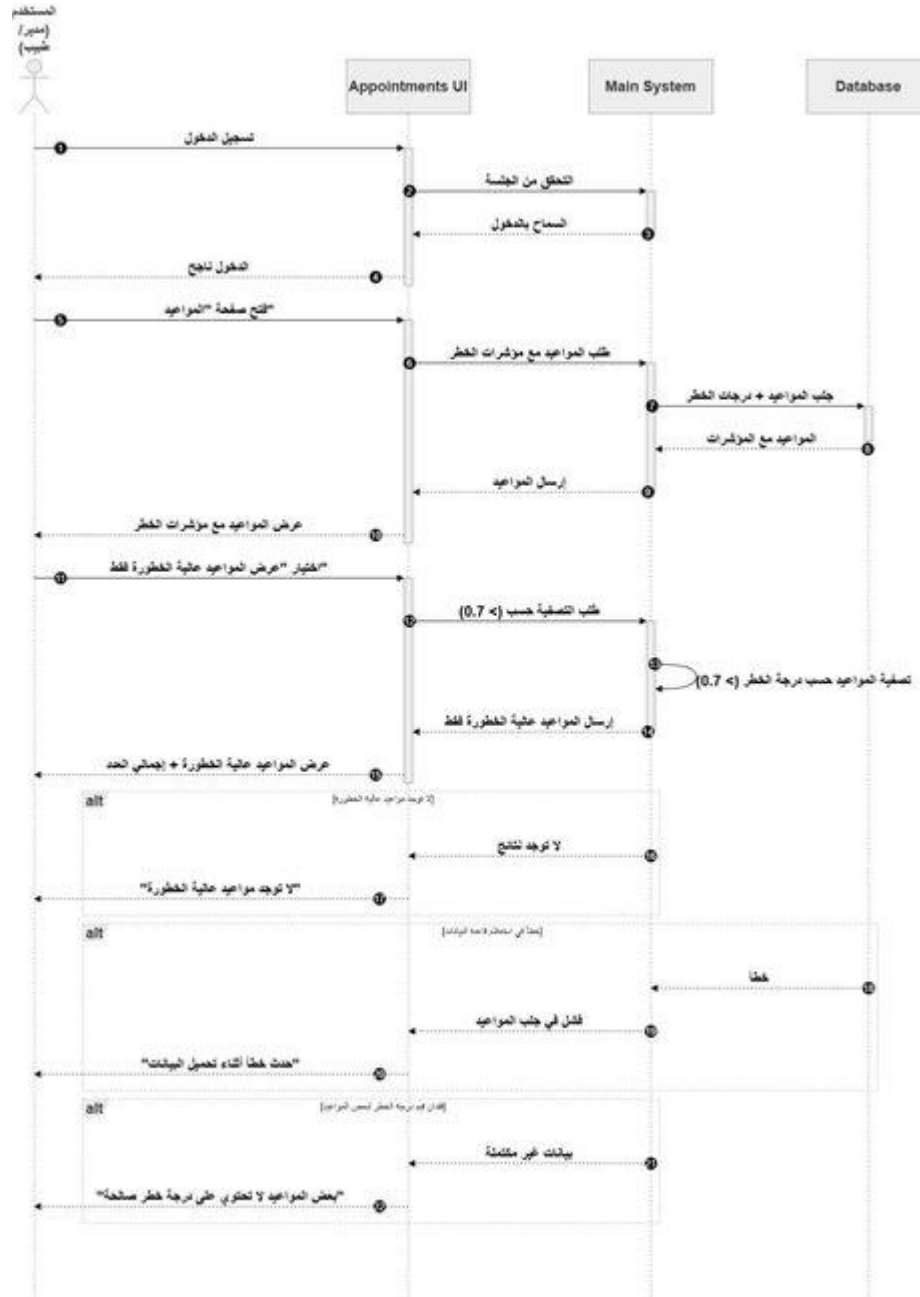
مخطط تسلسل المدير: عرض قائمة الموردين (View Suppliers List – UC-A-06)



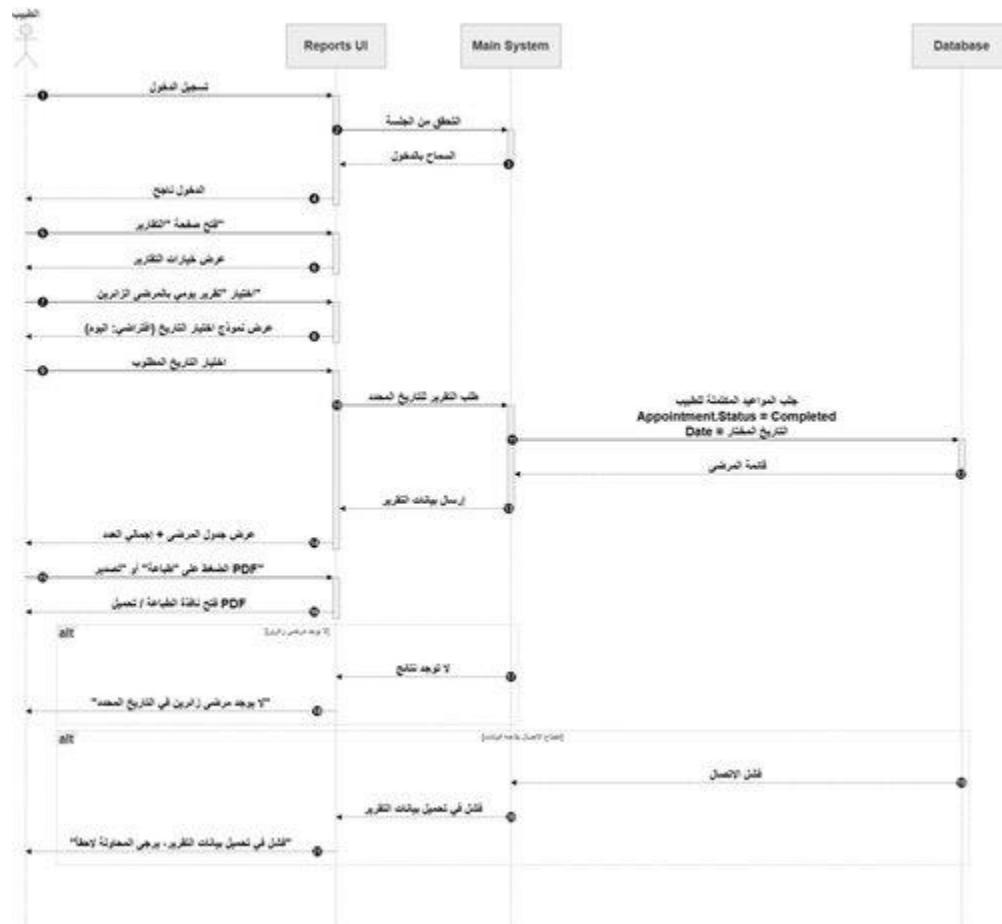
مخطط تسلسل المدير: إدارة قائمة الأسعار والخدمات (Manage Price List – UC-A-07)



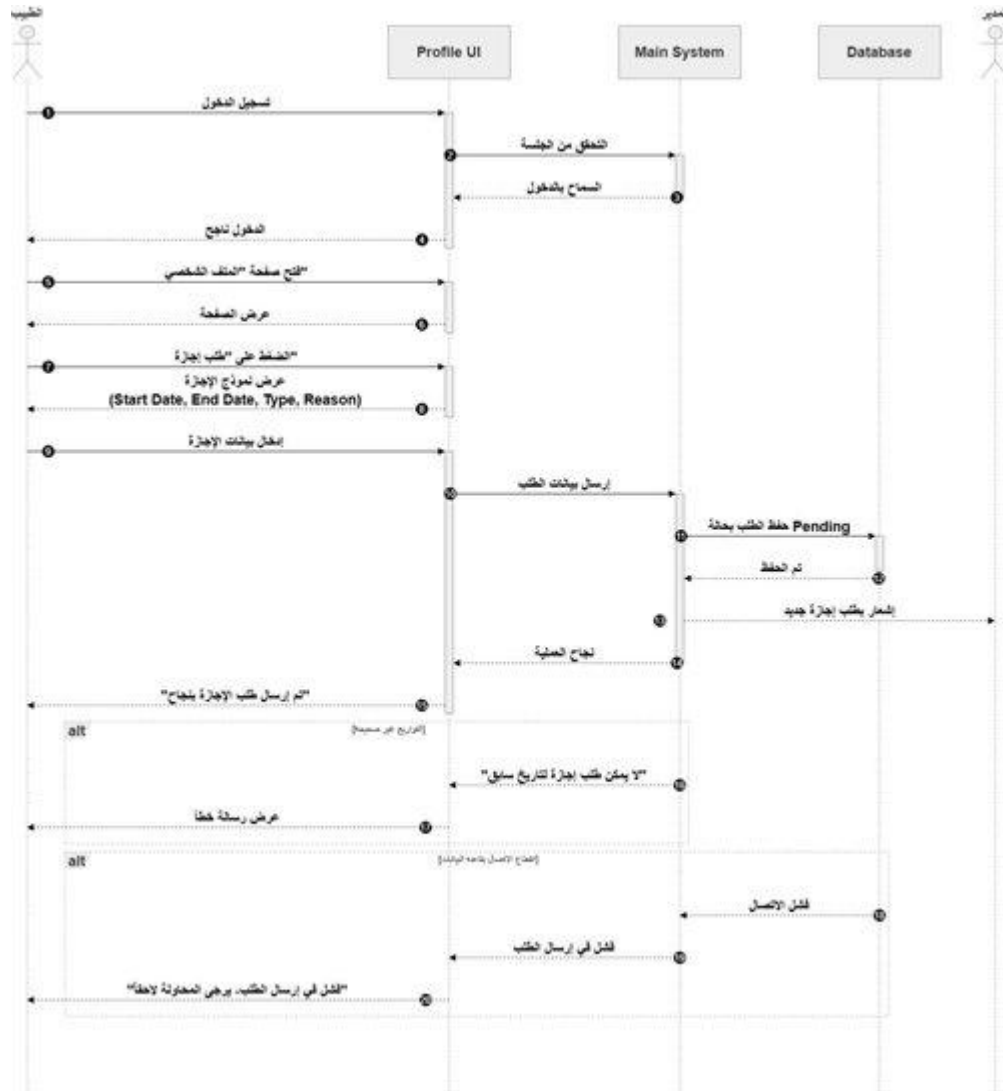
مخطط تسلسل المواعيد مع مؤشرات الذكاء الاصطناعي (Appointments with AI Prediction API)



مخطط تسلسل المواعيد مع مؤشرات الخطر (Appointments Risk Level View)



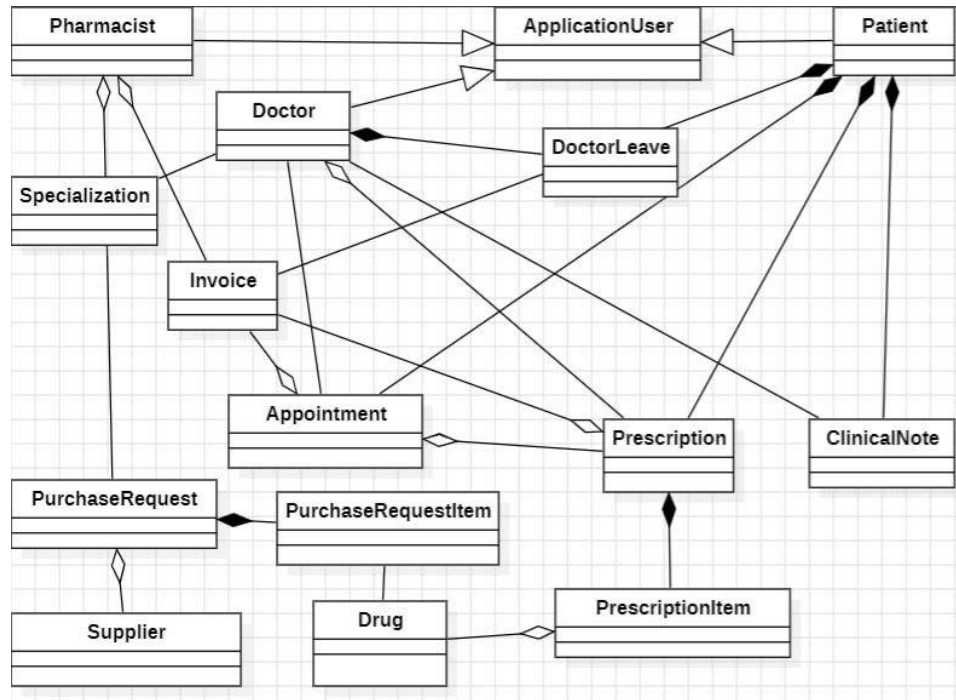
مخطط تسلسل طلب إجازة الطبيب (Doctor Leave Request)



مخطط تسلسل تقارير الطبيب (Doctor Daily Reports)

4-6-4 مخطط الفئات (Class Diagram)

يوضح مخطط الفئات التالي البنية الثابتة للنظام من خلال تصوير الفئات والعلاقات بينها وفق نمط OOP:



مخطط الفئات 1: Class Diagram لنظام HCMS

5-6-7 مصفوفة تتبع المتطلبات - الجديدة (Requirements Tracking Matrix)

ترتبط المصفوفة التالية المتطلبات الجديدة (الصيدلي والمدير) بمخططات UML وحالات الاختبار المرتبطة:

جدول RTM: مصفوفة تتبع المتطلبات الجديدة

الاختبار	Sequence	Activity	Use Case	الحالة	وصف المتطلب	معرف
تم	مخطط الصيدلي 1 تسلسل	مخطط نشاط الصيدلي 1	UC-Ph-01	مكتمل	تسجيل الدخول (صيدلي)	REQ01
تم	مخطط الصيدلي 2 تسلسل	مخطط نشاط الصيدلي 2	UC-Ph-02	مكتمل	عرض تفاصيل الوصفات	REQ02
تم	مخطط الصيدلي 3 تسلسل	مخطط نشاط الصيدلي 3	UC-Ph-03	مكتمل	إضافة دواء جديد	REQ03
تم	مخطط الصيدلي 4 تسلسل	مخطط نشاط الصيدلي 4	UC-Ph-04	مكتمل	تعديل دواء	REQ04
تم	مخطط الصيدلي 5 تسلسل	مخطط نشاط الصيدلي 5	UC-Ph-05	مكتمل	حذف دواء	REQ05
تم	مخطط الصيدلي 6 تسلسل	مخطط نشاط الصيدلي 6	UC-Ph-06	مكتمل	إدارة تنبيهات الصلاحية	REQ06
تم	مخطط الصيدلي 7 تسلسل	مخطط نشاط الصيدلي 7	UC-Ph-07	مكتمل	تقرير فواتير يومي	REQ07

الاختبار	Sequence	Activity	Use Case	الحالة	وصف المتطلب	معرف
تم	مخطط تسلسل الصيدلي 8	مخطط نشاط الصيدلي 8	UC-Ph-08	مكتمل	طلب شراء دواء	REQ08
ت	مخطط تسلسل المدير 4	مخطط نشاط المدير 4	UC-A-04	مكتمل	إضافة مورد جديد	REQ09
	مخطط تسلسل المدير 5	مخطط نشاط المدير 5	UC-A-05	مكتمل	حذف مورد	REQ10
	مخطط تسلسل المدير 6	مخطط نشاط المدير 6	UC-A-06	مكتمل	عرض قائمة الموردين	REQ11
	مخطط تسلسل المدير 7	مخطط نشاط المدير 7	UC-A-07	مكتمل	إدارة قائمة الأسعار	REQ12
	مخطط تسلسل المدير 8	مخطط نشاط المدير 8	UC-A-08	مكتمل	التقارير اليومية	REQ13

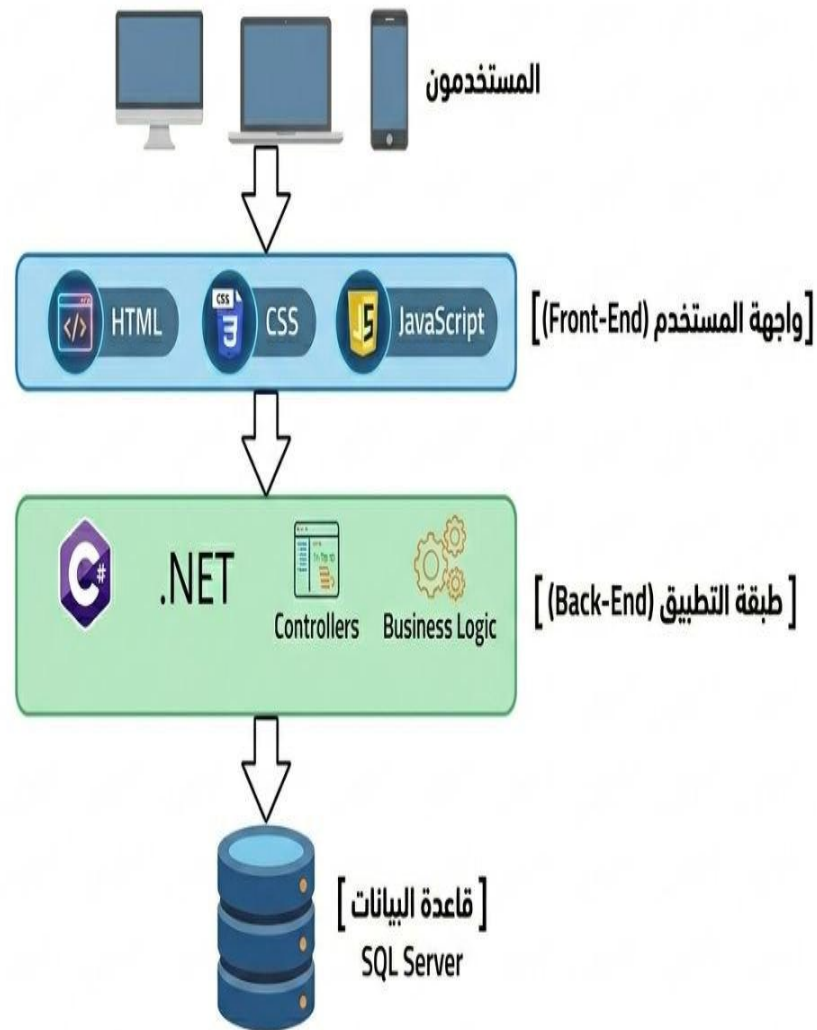
الفصل الخامس: التصميم المعماري للنظام

5.1 التصميم عالي المستوى (High-Level Design)

يهدف هذا القسم إلى توضيح البنية العامة للنظام وكيفية تفاعل مكوناته المختلفة لتحقيق الوظائف المطلوبة.

5.1.1 مخطط الكتل المعمارية (System Block Diagram)

يتكون النظام من عدة مكونات رئيسية تشمل واجهة المستخدم (Front-End) ، طبقة المعالجة (Back-End) ، وقاعدة البيانات. حيث تقوم واجهة المستخدم بإرسال الطلبات إلى الخادم، الذي بدوره يعالج البيانات ويتواصل مع قاعدة البيانات لإرجاع النتائج المناسبة للمستخدم.



الفصل الخامس

التصميم المعماري للنظام

5.2 نمط التصميم المعماري المستخدم

تم اعتماد نمط (Model-View-Controller (MVC في تصميم النظام، وذلك لكونه أحد أنماط التصميم المعمارية الأكثر استخدامًا في تطبيقات الويب الحديثة، لما يوفره من تنظيم منهجي لبنية التطبيق من خلال فصل المسؤوليات بين مكوناته الأساسية. حيث يقوم هذا النمط على تقسيم النظام إلى ثلاث طبقات رئيسية: Model التي تمثل طبقة البيانات ومنطق الأعمال، و View التي تمثل واجهة المستخدم وعرض البيانات، و Controller الذي يتولى معالجة طلبات المستخدم والتنسيق بين الطبقتين السابقتين.

يساهم هذا الفصل الواضح بين الطبقات في تحسين جودة الكود البرمجي وتقليل الترابط (Loose Coupling) بين مكونات النظام، مما يؤدي إلى تسهيل عمليات التطوير والصيانة وإمكانية تعديل أي جزء من النظام دون التأثير على الأجزاء الأخرى. كما يعزز هذا النمط قابلية التوسع (Scalability) في المستقبل، حيث يمكن إضافة ميزات جديدة أو تطوير وظائف قائمة بطريقة منظمة ومرنة دون إعادة هيكلة النظام بالكامل.

وبالتالي، فإن اعتماد نمط MVC في هذا المشروع يضمن تحقيق توازن بين الأداء وسهولة الإدارة، إضافة إلى رفع كفاءة الفريق أثناء عملية التطوير من خلال توزيع واضح للأدوار داخل التطبيق.

5.3 التصميم المعماري للنظام

يعتمد النظام على نمط التصميم المعماري (Model-View-Controller (MVC، والذي يُعد أحد أكثر الأنماط استخدامًا في تطوير تطبيقات الويب الحديثة، لما يوفره من تنظيم هيكلي واضح لفصل مكونات النظام وفقًا لمسؤولياتها.

في هذا النمط، تمثل Model طبقة منطق الأعمال وطبقة البيانات، حيث تتولى تعريف الكيانات (Entities) وإدارة العمليات المرتبطة بها، بالإضافة إلى التعامل مع قاعدة البيانات (SQL Server) وتنفيذ عمليات التخزين والاسترجاع ومعالجة البيانات.

أما View فتمثل طبقة العرض، وهي المسؤولة عن تقديم واجهات المستخدم وعرض البيانات بشكل تفاعلي ومنظم، بما يضمن تحسين تجربة المستخدم وتمكينه من التفاعل مع النظام بسهولة ووضوح.

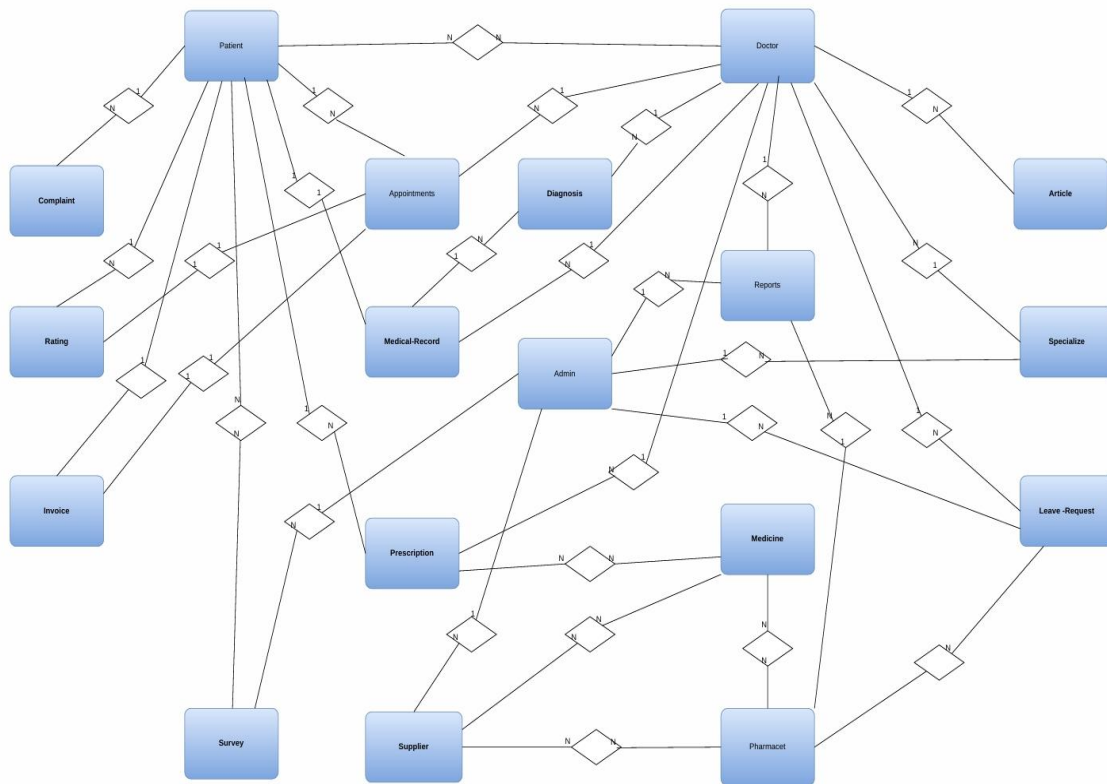
في حين تُعد Controller الطبقة الوسيطة التي تستقبل طلبات المستخدم، وتقوم بمعالجتها، ثم التنسيق بين طبقة Model و View من أجل تنفيذ المنطق المطلوب وإرجاع الاستجابة المناسبة.

إن اعتماد هذا النمط يساهم في تحقيق مبدأ فصل الاهتمامات (Separation of Concerns)، مما يؤدي إلى تحسين قابلية صيانة النظام، وتسهيل تطويره وتوسيعه مستقبلاً، بالإضافة إلى رفع جودة الكود البرمجي وتقليل الترابط بين مكونات التطبيق.

5.4 تصميم قاعدة البيانات

5.4.1 نموذج الكيانات والارتباطات (ER Diagram)

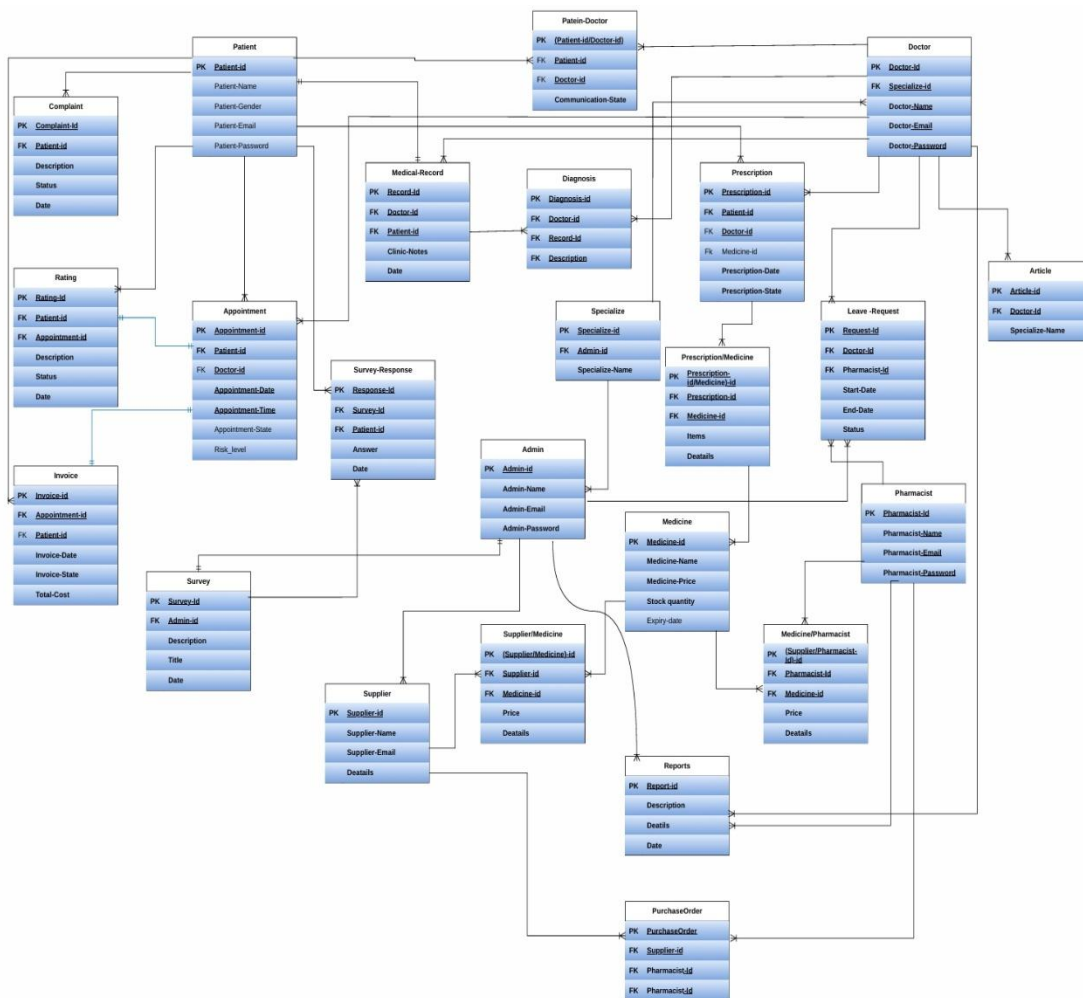
تم تصميم قاعدة البيانات باستخدام نموذج الكيانات والعلاقات (ERD) الذي يوضح الكيانات الأساسية مثل: المستخدمين، المرضى، المواعيد، والسجلات الطبية، بالإضافة إلى العلاقات فيما بينها.



مخطط ERD 2: مخطط الجداول (Table Diagram)

5.4.2 المخطط العلائقي (Relational Schema)

تم تحويل نموذج ER إلى جداول علائقية في قاعدة بيانات SQL Server، حيث يحتوي كل جدول على مجموعة من الحقول المرتبطة بنوع البيانات المناسب لضمان سلامة التخزين وكفاءة الاستعلامات



مخطط 2 ER: مخطط الجداول (Table Diagram)

5.5 القيود والمفاتيح (Constraints & Keys)

تم تطبيق مجموعة من القيود لضمان تكامل البيانات، وتشمل:

- المفتاح الأساسي (Primary Key) لتعريف كل سجل بشكل فريد.
- المفتاح الخارجي (Foreign Key) لربط الجداول ببعضها.
- قيود الفريدة (Unique) لمنع التكرار.
- قيود التحقق (Check) لضمان صحة البيانات المدخلة.

5.6 تصميم الواجهات الخارجية (APIs)

تصميم الأمان والصلاحيات

تم تطبيق نظام أمان يعتمد على التحكم بالصلاحيات (Role-Based Access Control - RBAC)، حيث تم تحديد أدوار المستخدمين مثل: مدير النظام، الطبيب، والمريض.

كما تم استخدام آلية مصادقة آمنة لتسجيل الدخول، بهدف حماية البيانات وضمان وصول المستخدمين فقط إلى الوظائف المسموح بها لهم.

الفصل السادس: بيئة التنفيذ والأدوات

6.1 الأدوات البرمجية والمكتبات المستخدمة

اعتماد بيئة تطوير متكاملة Visual Studio لتطوير النظام، نظرًا لما توفره من أدوات متقدمة لدعم عملية البرمجة، التصحيح (Debugging)، وإدارة المشاريع البرمجية بكفاءة عالية. كما تم استخدام SQL Server كنظام إدارة قواعد بيانات (DBMS)، لما يتميز به من موثوقية وأداء عالٍ في تخزين البيانات وإدارتها بشكل آمن ومنظم. بالإضافة إلى ذلك، تم الاستفادة من مجموعة من الأدوات المساندة التي تدعم عمليات التطوير والاختبار وتحسين جودة النظام.

6.2 بيئة التطوير وبنية المشروع

تم تطوير النظام ضمن بيئة تشغيل Windows، مع تنظيم هيكلية المشروع وفق نمط (MVC) Model-View-Controller لضمان وضوح البنية وسهولة الصيانة. وقد تم تقسيم المشروع إلى عدة مجلدات رئيسية، تشمل:

Controllers: لمعالجة طلبات المستخدم والتحكم بتدفق العمليات.

Models: لتمثيل البيانات ومنطق الأعمال والتعامل مع قاعدة البيانات.

Views: لعرض واجهات المستخدم والتفاعل مع النظام.

Data Access Layer: لإدارة عمليات الاتصال بقاعدة البيانات وتنفيذ الاستعلامات.

هذا التنظيم يساهم في تحقيق فصل واضح بين مكونات النظام، مما يعزز قابلية التطوير والتوسع المستقبلي.

لغات البرمجة وإطارات العمل

تم استخدام لغة البرمجة C# باعتبارها اللغة الأساسية لتطوير منطق النظام، وذلك ضمن إطار العمل ASP.NET MVC الذي يوفر بنية منظمة لتطوير تطبيقات الويب. كما تم استخدام تقنيات الواجهة الأمامية HTML, CSS, JavaScript لتصميم وتطوير واجهات المستخدم وتحسين تجربة الاستخدام من حيث التفاعل وسهولة الاستخدام.

6.3 مخدّم قاعدة البيانات ونظام التشغيل

تم اعتماد SQL Server كنظام إدارة قاعدة البيانات الرئيسي، حيث يعمل على نظام تشغيل Windows. وقد تم اختيار هذا النظام لما يوفره من استقرار عالٍ، وقدرة على معالجة كميات كبيرة من البيانات بكفاءة، بالإضافة إلى دعمه لخصائص الأمان وإدارة الوصول إلى البيانات بشكل متقدم.

6.4 المكتبات والتبعيات

يعتمد المشروع على مكتبات إطار عمل .NET / NET Framework القياسية، بالإضافة إلى مجموعة من حزم NuGet التي تم استخدامها لتوسيع وظائف النظام، مثل تحسين الاتصال بقاعدة البيانات، وإدارة العمليات البرمجية، ودعم الوظائف الإضافية اللازمة لتطوير التطبيق.

6.5 أدوات إدارة الإصدارات والسيطرة المصدريّة

تم استخدام نظام التحكم بالإصدارات Git بالتكامل مع منصة GitHub، وذلك بهدف تتبع التغييرات في الكود البرمجي، وإدارة الإصدارات المختلفة من النظام بشكل منظم. كما ساهم ذلك في تسهيل العمل التعاوني بين أعضاء الفريق، وتقليل التعارضات أثناء دمج التعديلات.

6.6 أدوات التكامل والنشر المستمر (CI/CD)

تم تطبيق مفهوم التكامل المستمر (Continuous Integration) بشكل مبسط باستخدام GitHub Actions، حيث يتم تنفيذ عمليات بناء (Build) واختبار تلقائية للتأكد من سلامة التعديلات قبل دمجها في الفرع الرئيسي، مما يساهم في تحسين جودة النظام وتقليل الأخطاء.

6.7 بيئة الاختبار (Testing Environment)

تم تنفيذ عمليات الاختبار في بيئة محلية (Local Environment) باستخدام Visual Studio، مع تشغيل النظام على localhost وربطه بقاعدة بيانات SQL Server. وقد أتاح ذلك التحقق من وظائف النظام بشكل آمن قبل عملية النشر الفعلي، مما ساعد في اكتشاف الأخطاء مبكرًا وضمان استقرار الأداء.

الفصل السابع: التنفيذ البرمجي

7.1 مقدمة

يُعدّ فصل التنفيذ البرمجي المرحلة التي تتحول فيها المخططات والتصاميم إلى كود برمجي فعلي قابل للتشغيل. وفي هذا الفصل، يتم عرض هيكل الملفات والمشروع، وشرح أبرز الوحدات البرمجية المنقّدة، مع تقديم نماذج من الكود وصور من واجهات النظام. ويُبنى النظام وفق نمط MVC باستخدام ASP.NET وSQL Server.

7.2 هيكل الملفات والمشروع

تم تنظيم المشروع وفق الهيكلية التالية التي تعكس نمط MVC بوضوح:

- مجلد Controllers: يحتوي على جميع متحكمات النظام مثل PatientController، DoctorController، AdminController، وAuthController.
- مجلد Models: يضم نماذج البيانات مثل Patient، Doctor، Appointment، Prescription، Medicine، وInvoice.
- مجلد Views: يحتوي على ملفات العرض مقسّمة حسب كل متحكم (/Patient، /Doctor، /Admin، /Shared).
- مجلد Data: يضم طبقة الوصول للبيانات (DbContext) والمهاجرات (Migrations).
- ملفات التكوين: appsettings.json لإعدادات قاعدة البيانات والتطبيق.

7.3 تنفيذ الوحدات النمطية الرئيسية

7.3.1 وحدة إدارة الحسابات والمصادقة

تم تنفيذ وحدة المصادقة باستخدام ASP.NET Identity مع تخصيص للأدوار (Roles) لتمييز بين المريض والطبيب والمدير. تعتمد عملية تسجيل الدخول على التحقق من البريد الإلكتروني وكلمة المرور المشفرة، مع إنشاء جلسة المستخدم (Session) فور نجاح المصادقة وتوجيهه للصفحة الخاصة بدوره.

7.3.2 وحدة إدارة المواعيد

تتضمن وحدة إدارة المواعيد منطق التحقق من صحة التاريخ (لا يمكن حجز موعد في الماضي)، وعرض المواعيد المتاحة، وحفظها في جدول Appointments في قاعدة البيانات. كما تتضمن ميزة إعادة الجدولة والإلغاء مع تحديث حالة الموعد تلقائيًا.

7.3.3 وحدة السجل الطبي

تتيح هذه الوحدة للأطباء الوصول إلى التاريخ المرضي الكامل للمريض، بما يشمل التشخيصات السابقة والوصفات الطبية والملاحظات السريرية. ويتم تخزين كل إدخال مرتبطاً بمعرف المريض ومعرف الطبيب وتاريخ الإدخال.

7.3.4 وحدة الصيدلانية والمخزون

تدير هذه الوحدة مخزون الأدوية من خلال عمليات الإضافة والتعديل والحذف مع التحقق من صحة البيانات. كما تتضمن منطق التحقق من تاريخ الصلاحية وإرسال تنبيهات للمدير عند اقتراب انتهاء صلاحية الأدوية.

4.7 تنفيذ طبقة الوصول إلى البيانات

تم استخدام Entity Framework Core > ORM (Object-Relational Mapper) للتعامل مع قاعدة البيانات. يوفر هذا الإطار إمكانية كتابة استعلامات LINQ بدلاً من SQL المباشر، مما يُحسّن أمان الكود ويُقلّل من مخاطر SQL Injection. وتُعرّف العلاقات بين الجداول ضمن ApplicationDbContext باستخدام Fluent API.

7.5 تنفيذ طبقة الأعمال (Business Logic)

يتضمن منطق الأعمال التالي في النظام:

- التحقق من عدم تضارب المواعيد: يتحقق النظام من عدم وجود موعد مسبق للطبيب في نفس التاريخ والوقت قبل تأكيد الحجز.
- منطق الأولوية في عرض المواعيد: تُعرض مواعيد اليوم أولاً ثم المواعيد المستقبلية مرتبةً تصاعدياً حسب التاريخ.
- التحقق من صحة المدخلات: يتم التحقق من صحة جميع الحقول قبل الحفظ في قاعدة البيانات مع إرسال رسائل خطأ واضحة للمستخدم.
- منطق تنبيه انتهاء الصلاحية: يتحقق النظام عند كل دخول للمدير من وجود أدوية تنتهي صلاحيتها خلال 30 يوماً ويعرض تنبيهاً.

7.6 تنفيذ واجهة المستخدم (UI)

تم تصميم واجهات المستخدم باستخدام Bootstrap 5 وHTML5 وCSS3 وJavaScript لضمان واجهة متجاوبة وسهلة الاستخدام. وتشمل الواجهات المنقّدة:

- صفحة تسجيل الدخول: نموذج بسيط بحقلين (بريد إلكتروني وكلمة مرور) مع رسائل خطأ واضحة.
- صفحة حجز الموعد: قائمة منسدلة لاختيار الطبيب، ومنتقي التاريخ والوقت (Date Picker)، مع زر تأكيد الحجز.

- صفحة السجل الطبي: عرض منظم للتشخيصات والوصفات مرتبةً من الأحدث للأقدم مع إمكانية عرض التفاصيل.
- صفحة إدارة المخزون: جدول تفاعلي يعرض الأدوية مع أزرار التعديل والحذف، وحقل بحث سريع.

7.7 الخلاصة

تم تنفيذ نظام HCMS بنجاح وفق نمط MVC باستخدام ASP.NET و SQL Server، مع الالتزام بمبادئ فصل المسؤوليات وتنظيم الكود. وقد تضمن التنفيذ جميع الوحدات الرئيسية المحددة في المتطلبات، من إدارة الحسابات والمواعيد والسجلات الطبية إلى الصيدلية والفوترة.

الفصل الثامن: الاختبار والتقييم

8.1 مقدمة

يهدف فصل الاختبار إلى إثبات أن نظام HCMS يعمل وفق المتطلبات المحددة وبمستوى جودة مقبول. وقد تم إجراء أنواع متعددة من الاختبارات تشمل اختبارات الوحدة واختبارات التكامل واختبارات النظام، وذلك بهدف اكتشاف أي أخطاء ومعالجتها قبل تسليم النظام.

8.2 خطة الاختبار

تُحدد خطة الاختبار ما يلي:

- نطاق الاختبار: جميع الوحدات الوظيفية للنظام (حسابات المستخدمين، المواعيد، السجل الطبي، الصيدلية).
- بيئة الاختبار: بيئة محلية (localhost) باستخدام Studio Visual مع قاعدة بيانات SQL Server.
- أدوات الاختبار: الاختبار اليدوي عبر المتصفح مع توثيق النتائج في جداول منظمة.
- معايير القبول: نجاح جميع حالات الاختبار الأساسية بنسبة 100%، واستجابة النظام ضمن أقل من 3 ثوانٍ.

8.3 أنواع الاختبارات المُجرّاة

8.3.1 اختبارات الوحدة (Unit Testing)

تم اختبار كل وحدة برمجية بشكل منفصل للتحقق من صحة منطق الأعمال، بما يشمل: التحقق من صحة التواريخ في حجز المواعيد، والتحقق من صحة كلمات المرور، وتنسيق البيانات المدخلة.

8.3.2 اختبارات التكامل (Integration Testing)

تم التحقق من صحة التفاعل بين مكونات النظام المختلفة، وتشمل: التكامل بين طبقة الواجهة (View) والمتحكم (Controller)، والتكامل بين المتحكم وقاعدة البيانات عبر Entity Framework.

8.3.3 اختبارات النظام (System Testing)

تم اختبار النظام بالكامل كوحدة متكاملة من خلال تنفيذ سيناريوهات استخدام حقيقية (End-to-End Testing)، بما يشمل: دورة حجز موعد كاملة، ودورة إنشاء وصفة طبية كاملة، وإدارة المخزون.

8.3.4 اختبارات قابلية الاستخدام (Usability Testing)

تم تقييم سهولة استخدام الواجهات من خلال مراجعة تصاميم الصفحات والتأكد من وضوح الرسائل وسهولة التنقل بين الصفحات، مع التحقق من وضوح رسائل الخطأ ومطابقتها للحالات المختلفة.

8.4 نتائج الاختبار

8.5 تحليل نتائج الاختبار

8.6 مقاييس الجودة

8.7 الخلاصة